

REINFORCEMENT LEARNING TUTORIAL

从理论到 PyTorch 实战 · 全栈深度算法精讲

强化学习 全面教程

覆盖 MDP、Q-Learning、DQN、Policy
Gradient、

Actor-Critic、PPO、SAC、Rainbow 到 RLHF

本教程系统讲解强化学习的数学基础、核心算法与工程实践，配套 12 份基于 PyTorch + Gymnasium 的可运行代码。从贝尔曼方程到 PPO、SAC，再到 RLHF 与离线 RL，构建从入门到前沿的完整知识体系。

PyTorch 2.x

Gymnasium

14 章

含代码

VERSION v1.0 · 2026.08

AUDIENCE ML 工程师 · 研究生 · 算法爱好者

CODE rl_tutorial_code/ 目录下 12 份脚本

目录

第一章 强化学习入门与核心概念	5
1.1 什么是强化学习	5
1.2 核心概念与术语	5
1.3 与监督学习、无监督学习的对比	6
1.4 RL 的分类体系	6
1.5 经典应用场景	6
1.6 第一个 RL 程序：环境交互循环	7
第二章 数学基础：MDP 与贝尔曼方程	9
2.1 MDP 的形式化定义	9
2.2 回报与价值函数	10
2.3 贝尔曼方程	11
2.4 FrozenLake：MDP 的实例分析	12
第三章 基于值的方法：动态规划	13
3.1 策略评估	14
3.2 策略改进	14
3.3 策略迭代	15
3.4 值迭代	15
3.5 DP 的局限与现代意义	16
第四章 蒙特卡洛方法	18
4.1 MC 预测	18
4.2 MC 控制	19
4.3 Blackjack 实战	19
4.4 MC 的优缺点	20
第五章 时序差分学习：SARSA 与 Q-Learning	21
5.1 TD(0) 预测	21
5.2 SARSA：on-policy TD 控制	22

5.3 Q-Learning: off-policy TD 控制	23
5.4 经典对比: CliffWalking	24
5.5 期望 SARSA 与算法选择	24
5.6 TD(λ): 在 MC 与 TD 之间插值	25
第六章 函数逼近与 DQN	26
6.1 函数逼近基础	26
6.2 DQN 的三大创新	26
6.3 DQN 完整算法	27
6.4 训练技巧与超参数	28
6.5 DQN 的局限	29
第七章 策略梯度方法: REINFORCE	30
7.1 策略目标函数	30
7.2 策略梯度定理	30
7.3 REINFORCE 算法	31
7.4 减小方差: 基线与优势	32
7.5 REINFORCE 的局限与改进方向	33
第八章 Actor-Critic 与 A2C/A3C	34
8.1 AC 框架的核心思想	34
8.2 A2C: 同步 Actor-Critic	35
8.3 A3C: 异步并行	35
8.4 GAE: 广义优势估计	35
8.5 A2C 实现要点	37
第九章 PPO: 现代 RL 工业标准	37
9.1 从 TRPO 到 PPO	37
9.2 PPO-Clip 损失函数	38
9.3 PPO 完整目标	38
9.4 PPO 训练流程	39
9.5 超参数选择	40
9.6 PPO 的工程实践	41

第十章 SAC 与连续控制	41
10.1 最大熵 RL：把探索写进目标函数	42
10.2 SAC 三大创新：把最大熵 RL 落地	43
10.3 SAC 完整算法	44
10.4 SAC 在 Pendulum 上的表现	45
10.5 SAC vs PPO vs DDPG/TD3	46
本章小结与下一章预告	47
第十一章 Rainbow DQN 与改进技巧	47
11.1 Double DQN：解决 Q 值过估（问题 1）	47
11.2 Dueling Network：分离 V 与 A（问题 2）	48
11.3 Prioritized Experience Replay（问题 3）	49
11.4 其他三项改进与数值实例	50
11.5 实战与对比	51
本章小结与下一章预告	51
第十二章 前沿主题：RLHF、DPO、世界模型、离线 RL、MARL	52
12.1 RLHF：驱动大模型对齐	52
12.2 DPO：直接偏好优化	52
12.3 世界模型：Dreamer 系列	55
12.4 离线 RL：从固定数据集学习	55
12.5 多智能体 RL：MARL	55
12.6 大模型时代的 RL 新方向	56
第十三章 算法对比与工程实践	56
14.1 算法选择决策树	56
14.2 算法综合对比	56
14.3 超参数经验	57
14.4 训练曲线分析	58
14.5 工程实践要点	58
14.6 算法对比的可视化思维	58

第十四章 调试、调参与常见问题排查 59

14.1 训练不收敛：可能的原因	59
14.2 Q 值爆炸	59
14.3 策略熵过早坍缩	59
14.4 Gymnasium 版本兼容	60
14.5 GPU 显存爆炸	60
14.6 探索-利用失衡	60
14.7 评估与对比的常见误区	61
14.8 调试 checklist	61

第一章 强化学习入门与核心概念

强化学习（Reinforcement Learning, RL）是机器学习的三大范式之一，与监督学习和无监督学习并列。其核心思想是：智能体（Agent）通过与环境（Environment）反复交互，根据环境反馈的奖励信号（Reward）不断调整自身行为策略，最终学会在复杂不确定环境下做出能最大化长期累积奖励的决策。这种“试错 + 反馈”的范式天然适合于序列决策问题——从棋类博弈、机器人控制到推荐系统、自动驾驶，再到近年来驱动 ChatGPT 等大语言模型对齐人类偏好的 RLHF 技术，强化学习无处不在。

本章将系统介绍强化学习的基本术语、与其他机器学习范式的区别、问题分类以及典型应用场景，帮助读者建立整体认知框架。后续章节会在此基础上逐步深入到具体算法和工程实现。

1.1 什么是强化学习

强化学习的本质是一个“决策优化”问题：智能体在时刻 t 观察到环境状态 s_t ，依据当前策略 π 选择动作 a_t ，环境接收动作后转移到新状态 s_{t+1} 并返回奖励 r_{t+1} 。智能体的目标是学习一个策略 π^* ，使得在长期交互中获得的累积奖励期望最大。这种“延迟反馈”特性是 RL 与监督学习最大的差异——监督学习中每个样本都有即时的“正确答案”，而 RL 中智能体可能要等多步之后才知道当初的某个动作是否正确。

强化学习的另一个核心特征是“探索与利用”（Exploration vs. Exploitation）的权衡。智能体既需要利用已知的高奖励行为（exploitation），又需要尝试未知行为以发现潜在更优策略（exploration）。这个困境在监督学习中并不存在，但在 RL 中几乎是所有算法都必须显式或隐式处理的问题。常见的探索策略包括 ϵ -greedy、Boltzmann 探索、UCB 以及基于噪声网络的方法等。

1.2 核心概念与术语

在深入算法之前，必须先掌握强化学习的基本词汇。以下术语贯穿全书后续章节。

- **智能体（Agent）**：决策主体，负责观察状态、选择动作、更新策略。
- **环境（Environment）**：智能体之外的万物，定义状态转移规则与奖励函数。
- **状态（State, s ）**：环境在某时刻的描述，可以是完全可观察的（如棋盘），也可以是部分可观察的（如打牌时不知道对手手牌）。
- **动作（Action, a ）**：智能体可执行的操作，可分为离散（左/右）或连续（速度大小）。
- **奖励（Reward, r ）**：环境对动作的即时反馈标量，是智能体优化的唯一信号。
- **策略（Policy, π ）**：从状态到动作的映射，可以是确定性的 $a = \pi(s)$ ，也可以是随机的 $\pi(a|s)$ 。
- **轨迹（Trajectory）**：一个回合内状态-动作-奖励序列 $\tau = (s_0, a_0, r_1, s_1, a_1, \dots)$ 。
- **回报（Return, G_t ）**：从时刻 t 起的累积折扣奖励 $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$ 。
- **价值函数（Value Function）**：状态或状态-动作对的长期期望回报，是 RL 评估的核心量。
- **模型（Model）**：环境的动力学描述 $P(s'|s,a)$ 与奖励函数 $R(s,a)$ 。

1.3 与监督学习、无监督学习的对比

理解 RL 的最好方式是与熟悉的监督学习对比。监督学习有标注好的 (x, y) 对，模型直接学习从输入到输出的映射；而 RL 中没有"正确动作"标签，智能体只能通过尝试不同动作并观察奖励来间接学习。监督学习的损失通常是预测与标签的差异，而 RL 的目标函数是期望累积奖励。此外，监督学习的样本通常是独立同分布的 (i.i.d.)，但 RL 中智能体产生的数据具有强时间相关性，这正是经验回放 (Experience Replay) 等技术存在的原因。

与无监督学习相比，RL 有明确的优化目标（奖励），而非寻找数据的隐藏结构。三者之间的关系可以这样概括：监督学习是"教模型做什么"，无监督学习是"让模型发现规律"，而强化学习是"让模型自己学会做什么"。这种自驱动特性使得 RL 在很多复杂决策场景中独树一帜。

1.4 RL 的分类体系

强化学习算法可从多个维度分类，掌握这些分类有助于在选择算法时快速定位。

1.4.1 按是否学习环境模型分类

- **基于模型 (Model-based)**：先学习环境模型 $P(s'|s,a)$ ，再用模型规划。代表：Dyna、MuZero、Dreamer。
- **无模型 (Model-free)**：不显式学习模型，直接从经验中学习。代表：DQN、PPO、SAC。

基于模型方法样本效率高（可"想象"经验），但模型误差会被放大；无模型方法更通用但样本需求大。现代 RL 在低数据场景（如机器人）倾向基于模型，在高维复杂场景（如 Atari）倾向无模型。

1.4.2 按学习的对象分类

- **基于值 (Value-based)**：学习 $Q(s,a)$ ，间接得到策略。代表：Q-Learning、DQN。适合离散动作。
- **基于策略 (Policy-based)**：直接参数化策略 $\pi_{\theta}(a|s)$ 。代表：REINFORCE、TRPO。适合连续动作。
- **Actor-Critic**：同时学策略与价值。代表：A2C、PPO、SAC。集大成者。

1.4.3 按策略更新方式分类

- **On-policy**：用当前策略产生的数据更新当前策略。代表：SARSA、REINFORCE、PPO。
- **Off-policy**：可用其他策略产生的数据更新当前策略。代表：Q-Learning、DQN、SAC。

Off-policy 通常样本效率更高（数据可重复使用），但训练可能不稳定；On-policy 更稳定但样本效率低。实践中：连续控制首选 PPO 或 SAC；离散控制首选 DQN 系列。

1.5 经典应用场景

强化学习的应用范围极其广泛，下面列举几个有代表性的领域，每个领域都有其独特的挑战与解决方案。

- **游戏 AI**：AlphaGo（围棋）、AlphaStar（星际争霸 2）、OpenAI Five（Dota 2），这些突破展示了 RL 在复杂策略博弈中的潜力。
- **机器人控制**：四足机器人行走、机械臂抓取、灵巧手操作。需要解决高维连续控制、稀疏奖励、Sim-to-Real 等难题。
- **自动驾驶**：车道保持、变道决策、路口通行。安全约束是关键挑战。
- **推荐系统**：新闻推荐、广告投放、商品推荐。RL 能优化长期用户参与度而非短期点击率。
- **大模型对齐（RLHF）**：用 RL 微调 LLM，使其行为符合人类偏好。这是 ChatGPT 成功的关键技术。
- **算法交易**：动态投资组合管理、做市策略、执行优化。需处理非平稳市场与极稀疏信号。

1.6 第一个 RL 程序：环境交互循环

理论讲完，让我们用一段最简单的代码建立直观感受。下面是使用 Gymnasium 库与 CartPole-v1 环境的最小交互循环。CartPole 是 RL 教学的"Hello World"：一根杆子立在小车上，目标是通过左右移动小车让杆子保持直立，每坚持一步得 1 分，最多 500 分（v1 版本）。


```

# — 导入 Gymnasium 库, 这是 OpenAI Gym 的维护分支 —
import gymnasium as gym
import numpy as np

# — 创建 CartPole-v1 环境 —
# v1 版本最多 500 步, v0 版本只有 200 步
env = gym.make("CartPole-v1")

# — 重置环境到初始状态 —
# Gymnasium 0.26+ 的 reset() 返回 (obs, info) 元组
# 用 seed=42 固定随机种子, 便于复现
obs, _ = env.reset(seed=42)

total_reward = 0.0 # 累计奖励计数器
terminated, truncated = False, False # 两个终止标志

# — 主循环: 直到回合结束 —
# terminated: 自然终止 (如杆倒了) —— 不应 bootstrap
# truncated: 外部截断 (如步数上限) —— 应该 bootstrap
while not (terminated or truncated):
    # 随机策略: 从动作空间均匀采样
    # CartPole 动作空间是 Discrete(2), 所以 randint(0, 2)
    action = np.random.randint(0, 2)

    # 执行动作, 返回 5 元组
    # obs: 新状态 (4维: 位置/速度/角度/角速度)
    # reward: 即时奖励 (每步 +1)
    # terminated: 是否自然终止
    # truncated: 是否被外部截断
    # info: 调试信息字典
    obs, reward, terminated, truncated, _ = env.step(action)

    total_reward += reward

print(f"随机策略得分: {total_reward}")
# 随机策略通常只得 10-30 分; 学完本教程后用 DQN/PP0 可达 500 分

```

这段代码揭示了 RL 的核心循环: 观察 (observe) → 决策 (act) → 执行 (step) → 反馈 (reward)。注意 Gymnasium 0.26+ 的 API 改动: reset() 返回 (obs, info) 元组, step() 返回 5 元组 (obs, reward, terminated, truncated, info), 其中 terminated 表示自然终止 (如杆倒了), truncated 表示因时间限制等外部条件终止。这是新手最常踩的坑。

[技巧] Gymnasium 是 OpenAI Gym 的维护分支, 2022 年起由 Farama Foundation 接管。旧代码中的 import gym 应迁移到 import gymnasium as gym。环境 API 也有重大变化, 详见第八章工程实践部分。

本教程配套代码包中, ch01_env_loop.py 是本章的完整可运行示例。建议读者先运行此脚本, 观察随机策略在 CartPole 上的表现, 再开始学习后续章节的算法。

本章我们建立了对强化学习的直觉认知——智能体、环境、状态、动作、奖励、策略, 这些是后续所有讨论的概念骨架。但仅有直觉是不够的: 要设计出可证明、可实现、可比较的算法, 我们必须把这些概念用精确的数学语言描述出来。

有了这些概念，下一章我们将用数学语言形式化地描述智能体与环境交互的规律——这就是马尔可夫决策过程（MDP）。MDP 是后续所有算法的基础：从动态规划到深度强化学习，都是求解 MDP 的不同方法。

第二章 数学基础：MDP 与贝尔曼方程

第一章我们了解了 RL 的基本概念——智能体、环境、奖励、策略。但要设计算法，还需要用数学精确描述这些概念，否则“最大化累积奖励”这种说法就只是一个口号。本章引入 MDP（马尔可夫决策过程）五元组，它是对 RL 问题的数学抽象：只要一个问题能写成 MDP，就可以用本书后续所有算法求解。

本章的目标是建立三件事：（1）MDP 的形式化定义；（2）价值函数的数学刻画；（3）连接价值函数与最优策略的“贝尔曼方程”。其中贝尔曼方程是后续所有算法——动态规划、蒙特卡洛、TD、DQN、PPO——的共同理论根基。

2.1 MDP 的形式化定义

我们一步步把第一章的直觉概念翻译成数学符号。MDP 由五元组 (S, A, P, R, γ) 定义，每个元素都对应第一章中我们已经熟悉的概念。

状态 S 是环境的描述。第一章说“智能体观察到环境状态 s_t ”——这里 S 就是所有可能状态的集合， $s \in S$ 表示某一时刻环境所处的情况。例如 FrozenLake 中 S 是 16 个网格编号，CartPole 中 S 是连续 4 维向量。状态必须包含做决策所需的全部信息，这就是“马尔可夫性”——稍后展开。

动作 A 是智能体的选择。在状态 s 下，智能体可以从动作集合 A 中选一个动作 $a \in A(s)$ 。动作可以是离散的（如 FrozenLake 的上下左右 4 个），也可以是连续的（如机械臂关节角度）。本章我们先聚焦离散动作，连续动作留到第十章策略方法讨论。

转移函数 P 描述环境如何响应。 $P(s'|s,a)$ 表示：在状态 s 执行动作 a 后，环境转移到 s' 的概率。注意 P 是概率而不是确定性映射——同样的 (s,a) 可能有多种不同的下一状态。例如 FrozenLake 的“冰面湿滑”设定下，选“向右”也只有 1/3 概率真的向右。 P 是环境本身的属性，智能体无法改变，但可以学习它（基于模型方法）或绕过它（无模型方法）。

奖励 R 是反馈。 $R(s,a)$ 或 $R(s,a,s')$ 表示在状态 s 执行动作 a （并转移到 s' ）后获得的期望奖励。奖励是环境的“评分”，是智能体优化目标的唯一信号来源。注意 RL 中奖励通常是稀疏且延迟的——例如 FrozenLake 只在到达终点给 +1，中间所有步奖励都是 0。如何把这种延迟的奖励“分配”给早期动作，是 RL 的核心难题之一。

折扣因子 γ 权衡当前与未来。 $\gamma \in [0, 1)$ 是一个标量，决定智能体有多重视未来的奖励。 $\gamma \rightarrow 1$ 表示“远视”（重视长远）， $\gamma \rightarrow 0$ 表示“近视”（只看眼前）。 γ 还有一个数学上的关键作用：当奖励有界且 $\gamma < 1$ 时，无限时序的累积回报是有界的，这保证了价值函数的良定义性。实践中 γ 通常取 0.9–0.99。

把上述五个元素合起来，MDP 用五元组 (S, A, P, R, γ) 完整刻画了一个 RL 问题。MDP 的核心假设是**马尔可夫性**：下一状态 s_{t+1} 和奖励 r_{t+1} 只依赖于当前状态 s_t 和动作 a_t ，而与更早的历史无关。用数学语言表述为：

$$P(s_{t+1} = s' | s_0, a_0, \dots, s_t, a_t) = P(s' | s_t, a_t)$$

这个假设极大地简化了问题：智能体不需要记住完整历史，只需要当前状态就能做出最优决策。当然，现实世界中很多问题并不严格满足马尔可夫性（如打牌时不知道对手手牌），此时可以通过引入“部分可观察 MDP”（POMDP）来处理，但底层算法仍基于 MDP。

2.2 回报与价值函数

MDP 定义了“游戏规则”，但还没说智能体要优化什么。RL 的优化目标很朴素：智能体的目标是最大化累积奖励，这个累积量叫**回报 G_t** 。我们从这里开始一步步推导价值函数。

第 1 步：写出回报的定义。 G_t 是从时刻 t 起所有未来奖励的折扣累加。先写出未折扣的形式： $G_t = r_{t+1} + r_{t+2} + r_{t+3} + \dots$ 但无限求和可能发散，所以用折扣因子 γ 加权未来奖励——越远的奖励权重越小：

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

第 2 步：发现 G_t 的递归结构。把上式第一项 r_{t+1} 提出来，剩下的部分恰好是 G_{t+1} 乘以 γ ：

$$G_t = r_{t+1} + \gamma \cdot (r_{t+2} + \gamma r_{t+3} + \dots) = r_{t+1} + \gamma G_{t+1}$$

这个“当前奖励 + $\gamma \times$ 下一时刻回报”的递归形式，是后续推导贝尔曼方程的钥匙。

第 3 步：直接优化 G_t 不方便，因为同一个状态后可能有许多不同的轨迹。考虑状态 s ：从 s 出发，由于 P 是随机的、 π 也可能随机，智能体可能走出 $G_t=10$ 的轨迹，也可能走出 $G_t=2$ 的轨迹。我们取**期望**，得到价值函数 $V^\pi(s)$ ——它衡量“在策略 π 下，状态 s 平均能拿到多少回报”。

$$V^\pi(s) = E_\pi[G_t | s_t = s]$$

这里“期望”是对什么取平均？答案是对**所有从 s 出发的随机性来源**取平均——包括策略 π 选动作的随机性、转移函数 P 的随机性、奖励 R 的随机性。这就是 V 的物理含义：在策略 π 下从 s 出发，反复跑很多次，平均得到的回报。

第 4 步：同理引入动作价值函数 Q 。有时我们不仅想知道“状态 s 有多好”，还想知道“在 s 下执行特定动作 a 有多好”。只需把“从 s 出发”细化为“在 s 选 a 后”，就得到 $Q^\pi(s,a)$ ：

$$Q^\pi(s,a) = E_\pi[G_t | s_t = s, a_t = a]$$

V 与 Q 的关系非常自然：在状态 s 下，按策略 π 选动作， Q 的期望就是 V ：

$$V^\pi(s) = \sum_a \pi(a|s) \cdot Q^\pi(s,a)$$

直观上， V 是“这个状态有多好”， Q 是“在这个状态下执行这个动作有多好”。基于值的算法（如 DQN）学 Q ，基于策略的算法直接学 π ，Actor-Critic 同时学两者。这个区分贯穿后续章节。

2.3 贝尔曼方程

价值函数 $V^\pi(s)$ 的定义是期望回报，但这个定义"不好算"——它要展开无限步的未来。贝尔曼方程的关键洞察是：利用 $G_t = r_{t+1} + \gamma G_{t+1}$ 这个递归结构，可以把 $V^\pi(s)$ 拆成"即时奖励 + 下一状态的价值"，从而把无限问题变成有限递归。下面我们一步步推导，不直接给结论。

2.3.1 贝尔曼期望方程的五步推导

第 1 步：从定义出发。 $V^\pi(s)$ 的定义是期望回报，而 $G_t = r_{t+1} + \gamma G_{t+1}$ ，所以：

$$V^\pi(s) = E_\pi[G_t | s_t = s] = E_\pi[r_{t+1} + \gamma G_{t+1} | s_t = s]$$

第 2 步：用期望的线性性拆开。期望有线性性，所以 $E[r_{t+1} + \gamma G_{t+1}] = E[r_{t+1}] + \gamma E[G_{t+1}]$ 。但这里要注意： r_{t+1} 和 G_{t+1} 都依赖于"在 s 选了什么动作"，所以要把"选动作"的随机性显式写出来。

第 3 步：用全期望公式展开"选动作"和"转移到哪里"两层随机性。在状态 s ，策略 π 以概率 $\pi(a|s)$ 选动作 a ；选定 a 后，环境以概率 $P(s'|s,a)$ 转移到 s' 。对这两层随机性取期望：

$$V^\pi(s) = \sum_a \pi(a|s) \cdot \sum_{s'} P(s'|s,a) \cdot [R(s,a,s') + \gamma \cdot E_\pi[G_{t+1} | s_{t+1} = s']]$$

这里 $R(s,a,s')$ 是在 (s,a,s') 这条转移上获得的即时奖励（已对奖励的随机性取了期望）。方括号里"剩下的部分"是"到达 s' 之后还能拿到的期望回报"。

第 4 步：识别内层期望就是 $V^\pi(s')$ 。注意 $E_\pi[G_{t+1} | s_{t+1} = s']$ 恰好就是 $V^\pi(s')$ 的定义！因为从 s' 出发的期望回报就是 $V^\pi(s')$ 。代入得到：

$$V^\pi(s) = \sum_a \pi(a|s) \cdot \sum_{s'} P(s'|s,a) \cdot [R(s,a,s') + \gamma \cdot V^\pi(s')]$$

第 5 步：这就是贝尔曼期望方程。它的物理意义非常清晰： $V^\pi(s)$ 被拆成了"按 π 选动作的期望" + "下一状态价值的折扣期望"。直观地说：当前状态的价值 = 即时奖励的期望 + $\gamma \times$ 下一状态价值的期望。这是一个递归关系—— V 依赖 V ，看似循环，但正是这种结构使得迭代求解成为可能（第三章动态规划的核心思路）。

同样的推导对 Q 也成立，只需多拆一层"下一动作 a' "：

$$Q^\pi(s,a) = \sum_{s'} P(s'|s,a) \cdot [R(s,a,s') + \gamma \cdot \sum_{a'} \pi(a'|s') \cdot Q^\pi(s',a')]$$

2.3.2 贝尔曼最优方程：把 $\sum_a \pi(a|s)$ 替换为 $\max_a$

贝尔曼期望方程描述的是"任意策略 π "的价值。但 RL 的终极目标是找**最优**策略 π^* 。一个策略是最优的，当且仅当它在每个状态下都选最优动作。直觉上：如果 π^* 在状态 s 已经是最优，那么"按 π^* 选动作"就等价于"直接选 Q 最大的动作"——于是 $\sum_a \pi(a|s) \cdot (\dots)$ 这个加权平均，就应该被替换为 $\max_a (\dots)$ 。

形式化地说：对最优策略 π^* ，我们有 $\pi^*(a|s) = 1$ 当 $a = \arg\max_a Q^*(s,a)$ ，其余动作为 0。代入贝尔曼期望方程，加权求和 $\sum_a \pi^*(a|s) \cdot Q^*$ 就退化为 $\max_a Q^*$ ：

$$V^*(s) = \max_a \sum_{s'} P(s'|s,a) \cdot [R(s,a,s') + \gamma \cdot V^*(s')]$$

$$Q^*(s,a) = \sum_{s'} P(s'|s,a) \cdot [R(s,a,s') + \gamma \cdot \max_a Q^*(s',a)]$$

这就是贝尔曼最优方程。它与贝尔曼期望方程的唯一区别是：把"按 π 加权求和"换成了"取最大值"。这个看似简单的替换意义深远：最优策略可以从 Q^* 直接推出 $\pi^*(s) = \operatorname{argmax}_a Q^*(s,a)$ 。这正是 Q-Learning 的理论基础——只要学到 Q^* ，就能得到最优策略。

[提示] 贝尔曼最优方程是非线性的（含 $\max$ 操作），不能直接解析求解。这就是为什么后续章节需要各种迭代算法——动态规划（已知 MDP 时）、蒙特卡洛（无模型时）、TD（单步更新）——它们本质上都是用不同方式逼近这个非线性方程的解。

☒ 具体计算实例：2 状态 MDP 上的贝尔曼方程求解

考虑一个最小 MDP：2 个状态 {A, B}，2 个动作 {留, 走}，折扣因子 $\gamma=0.9$ 。转移与奖励定义如下：

- 状态 A 执行"留"：100% 留在 A，奖励 +1
- 状态 A 执行"走"：100% 到 B，奖励 0
- 状态 B 执行"留"：100% 留在 B，奖励 +2
- 状态 B 执行"走"：100% 到 A，奖励 0

第 1 步：求均匀随机策略 $\pi(a|s)=0.5$ 下的 V^π

$$V^\pi(A) = 0.5 \times [1 + 0.9 \cdot V(A)] + 0.5 \times [0 + 0.9 \cdot V(B)]$$

$$V^\pi(B) = 0.5 \times [0 + 0.9 \cdot V(A)] + 0.5 \times [2 + 0.9 \cdot V(B)]$$

$$\text{化简：} V(A) = 0.5 + 0.45 \cdot V(A) + 0.45 \cdot V(B), \quad V(B) = 1.0 + 0.45 \cdot V(A) + 0.45 \cdot V(B)$$

$$\text{两式相减：} V(B) - V(A) = 0.5; \quad \text{代入解得 } V(A) \approx 7.25, V(B) \approx 7.75$$

第 2 步：求最优 V^*

$$V^*(A) = \max(1 + 0.9 \cdot V^*(A), 0 + 0.9 \cdot V^*(B)) = \max(1 + 0.9 \cdot V^*(A), 0.9 \cdot V^*(B))$$

$$V^*(B) = \max(0 + 0.9 \cdot V^*(A), 2 + 0.9 \cdot V^*(B)) = \max(0.9 \cdot V^*(A), 2 + 0.9 \cdot V^*(B))$$

分析：若 A 选"留"、B 选"留"，则 $V^*(A) = 1 + 0.9 \cdot V^*(A)$ ，解得 $V^*(A)=10$ ； $V^*(B) = 2 + 0.9 \cdot V^*(B)$ ，解得 $V^*(B)=20$ 。

验证：A 选"走"得 $0.9 \cdot 20 = 18 > 10$ ，所以 A 应选"走"！重新求解：

$$V^*(A) = 0.9 \cdot V^*(B), \quad V^*(B) = 2 + 0.9 \cdot V^*(B) \rightarrow V^*(B)=20, V^*(A)=18$$

最终： $V^*(A)=18, V^*(B)=20$ ，最优策略 $\pi^*(A)=\text{走}, \pi^*(B)=\text{留}$

启示：即使 2 状态 MDP 也能展示"短期诱惑 vs 长期收益"的权衡——A 选"留"得即时 +1，但选"走"能去 B 拿 +2/步。最优策略放弃短期奖励。

2.4 FrozenLake: MDP 的实例分析

为了让概念具体化，我们以 FrozenLake-v1 为例分析一个完整 MDP。FrozenLake 是 4×4 网格世界，目标是让智能体从起点 S 走到目标 G，途中避开冰洞 H。环境有 is_slippery 选项：开启时动作有 1/3 概率横向滑动（模拟冰面湿滑），使问题更具挑战性。

```
import gymnasium as gym

# — 创建 FrozenLake-v1 环境 —
# map_name="4x4": 4x4 网格世界
# is_slippery=True: 冰面湿滑，动作有 1/3 概率横向滑动
env = gym.make("FrozenLake-v1", map_name="4x4", is_slippery=True)

# — 查看 MDP 的完整转移函数 —
# env.unwrapped.P 是一个嵌套字典: P[state][action] = [(p, s_next, r, done), ...]
# 每个元素表示一个可能的转移:
# p: 转移概率
# s_next: 下一状态编号
# r: 奖励
# done: 是否终止
P = env.unwrapped.P

# 查看状态 0 执行动作 2 (向右) 的转移
print(P[0][2])
# 输出: [(0.333, 0, 0.0, False), (0.333, 1, 0.0, False), (0.333, 0, 0.0, False)]
# 含义解读:
# 1/3 概率原地不动 (状态 0 → 0)
# 1/3 概率向下到状态 1
# 1/3 概率回到状态 0
# 这就是"冰面湿滑"——想向右走，但可能滑向其他方向
```

Gymnasium 的 env.unwrapped.P 字段直接暴露了环境的 MDP，可以直接用于动态规划求解（见第三章）。但实际中绝大多数环境不会提供 P，必须通过交互学习。

配套代码 ch02_mdp_frozen_lake.py 实现了完整的策略评估代码，可以直接运行观察均匀随机策略下的状态价值函数。建议读者运行后对比不同 γ 值下的 V^π ，直观感受折扣因子的影响。

本章我们建立了 RL 的数学骨架：MDP 五元组刻画问题，价值函数衡量"好坏"，贝尔曼方程给出"当前价值 = 即时奖励 + 下一价值"的递归结构。这套理论框架清晰而优美——但有一个现实问题：贝尔曼方程需要完全已知 MDP（即 P 和 R）才能直接求解，而绝大多数实际问题中我们并不知道 P 和 R。

下一章我们介绍动态规划——在**已知 MDP**时如何高效求解贝尔曼方程。它本身在现实中很少直接可用，但它揭示了所有后续算法的思想原型：蒙特卡洛、TD、DQN 都是"在不知道 MDP 的情况下，用样本近似动态规划"的不同方案。

第三章 基于值的方法：动态规划

第二章我们推导了贝尔曼方程——它把"无限步的期望回报"分解成"即时奖励 + 下一状态价值"。这套理论很优雅，但有一个数学上的麻烦：贝尔曼方程的右边含有 $V^\pi(s')$ ，是**递归的**，不能像普通方程那样直接求出 V^π 。

本章介绍动态规划 (Dynamic Programming, DP) ——在**已知 MDP** (即 P 和 R 已知) 时, 通过迭代求解贝尔曼方程。DP 是 RL 的"祖先算法": 它在大多数实际问题中并不直接可用 (因为现实中几乎不知道 P 和 R), 但它揭示了所有后续算法的思想原型。理解了 DP, 再看蒙特卡洛、TD、DQN, 你会发现它们都是"在不知道 MDP 时如何近似 DP"的不同方案。

3.1 策略评估

策略评估 (Policy Evaluation) 要解决的问题非常具体: 给定一个策略 π , 求它的价值函数 V^π 。我们的起点是第二章推导的贝尔曼期望方程:

$$V^\pi(s) = \sum_a \pi(a|s) \cdot \sum_{s'} P(s'|s,a) \cdot [R(s,a,s') + \gamma \cdot V^\pi(s')]$$

问题: 右边含有 $V^\pi(s')$, 所以不能直接求解。但仔细看这个方程: 它说" V 的真值满足某种关系"。如果我们有一个"猜的 V "——记为 V_k ——把它代入右边, 得到的左边结果应该比 V_k 更接近真值。

这给我们一个迭代思路: 用第 k 轮的 V 估计第 $k+1$ 轮的 V 。具体地说: 把右边的 $V^\pi(s')$ 换成 $V_k(s')$, 左边就得到 $V_{k+1}(s)$:

$$V_{k+1}(s) = \sum_a \pi(a|s) \cdot \sum_{s'} P(s'|s,a) \cdot [R(s,a,s') + \gamma \cdot V_k(s')]$$

这叫**迭代策略评估**——反复用旧 V 算新 V , 直到收敛。理论上当 $k \rightarrow \infty$ 时 V_k 收敛到 V^π (这是压缩映射定理的结论, 证明从略)。实践中用一个小的阈值 θ (如 $1e-8$) 检测收敛: 当 $\max |V_{k+1}(s) - V_k(s)| < \theta$ 时停止。

代码上, 这只是一个三重循环: 遍历所有状态、所有动作、所有可能的下一状态, 计算期望。

```
def policy_eval(env, policy, gamma=0.99, theta=1e-8):
    # 参数: env=环境, policy=策略表[nS,nA], gamma=折扣, theta=收敛阈值
    nS = env.observation_space.n # 状态总数
    V = np.zeros(nS) # 初始化 V^pi(s) = 0

    while True: # 迭代直到收敛
        delta = 0 # 本轮最大变化量

        # 贝尔曼期望方程: V^pi(s) = sum_a pi(a|s) sum_{s'} P(s'|s,a) [R + gamma V(s')]
        for s in range(nS):
            v = 0
            for a, pi_a in enumerate(policy[s]): # 遍历所有动作
                for (p, s_next, r, done) in env.unwrapped.P[s][a]:
                    # (1-done): 终止状态不 bootstrap (不估值)
                    v += pi_a * p * (r + gamma * V[s_next] * (1 - done))
            delta = max(delta, abs(v - V[s])) # 跟踪最大变化
            V[s] = v # 更新 V(s)

        if delta < theta: # 所有状态变化都小于阈值 -> 收敛
            break
    return V
```

3.2 策略改进

策略评估给了我们 V^π ——但 π 可能不是最优的。一个自然的问题冒出来：**能否用 V^π 构造一个比 π 更好的策略？**

直觉：在状态 s ，如果某个动作 a 的"长期价值"比其他动作大，那我们就该选 a ，而不是按 π 选。这里"动作 a 的长期价值"正是 $Q^\pi(s,a)$ 。但问题是：我们只算了 V^π ，没算 Q^π ，怎么办？

关键： $Q^\pi(s,a)$ 可以从 V^π 直接算出来！因为 Q 的定义是"在 s 执行 a 后的期望回报"，而执行 a 后我们会到 s' ，从 s' 起就是按 π 走，所以从 s' 起的期望回报正是 $V^\pi(s')$ 。把所有可能的 s' 加权求和：

$$Q^\pi(s,a) = \sum_{s'} P(s'|s,a) \cdot [R(s,a,s') + \gamma \cdot V^\pi(s')]$$

有了 Q ，构造新策略就很直接——在每个状态选 Q 最大的动作：

$$\pi'(s) = \operatorname{argmax}_a Q^\pi(s,a) = \operatorname{argmax}_a \sum_{s'} P(s'|s,a) \cdot [R(s,a,s') + \gamma \cdot V^\pi(s')]$$

策略改进定理保证：新策略 π' 至少不比原策略 π 差，即 $V^{\pi'}(s) \geq V^\pi(s)$ 对所有 s 成立。如果 $\pi' = \pi$ （贪心策略没变化），则 π 已经是最优策略。这个定理是策略迭代收敛性的理论基础。

3.3 策略迭代

把策略评估和策略改进拼起来，就得到**策略迭代**（Policy Iteration）：评估 $\pi \rightarrow$ 改进得到 $\pi' \rightarrow$ 评估 $\pi' \rightarrow$ 改进得到 $\pi'' \rightarrow \dots$ 直到策略不再变化。这就像"先考试再调学习计划"的循环——评估是"考试"（测出当前策略的水平），改进是"调计划"（根据分数调整下一步该做什么）。

- 1. 初始化随机策略 π_0
- 2. 评估 π_k 得到 V^{π_k}
- 3. 改进得到 π_{k+1}
- 4. 若 $\pi_{k+1} = \pi_k$ 则停止，否则回到步骤 2

理论上策略迭代会在有限步内收敛到最优策略（对于有限 MDP）。但每次策略评估本身需要多轮迭代，整体开销较大。配套代码 `ch03_dp.py` 实现了完整的策略迭代，在 FrozenLake 上通常 2-3 次外循环即可收敛。

3.4 值迭代

策略迭代虽然能收敛，但有一个效率问题：**每次评估都要等 V 完全收敛才改进**，太慢了。一个观察：在策略评估的早期迭代中， V 的"形状"已经接近最优策略的 V^* ——没必要等到收敛。

值迭代的想法：不等了！把策略评估和策略改进合并成一个更新规则。怎么做？回顾第二章的贝尔曼最优方程：

$$V^*(s) = \max_a \sum_{s'} P(s'|s,a) \cdot [R(s,a,s') + \gamma \cdot V^*(s')]$$

对比策略评估的更新规则——它用 $\sum_a \pi(a|s)$ （按策略加权）；而贝尔曼最优方程用 $\max_a$ （直接取最优）。把贝尔曼最优方程直接作为迭代更新规则，就得到值迭代：

$$V_{k+1}(s) = \max_a \sum_{s'} P(s'|s,a) \cdot [R(s,a,s') + \gamma \cdot V_k(s')]$$

这一步既评估（计算期望）又改进（取 max），所以比策略迭代快——通常能少用很多次外循环。收敛后再通过一次策略改进 $\pi^*(s) = \operatorname{argmax}_a Q^*(s,a)$ 就得到最优策略。

```
def value_iteration(env, gamma=0.99, theta=1e-8):
    # 与策略评估的区别：用 max_a 替代  $\sum_a \pi(a|s)$ 
    # 每次更新隐式包含策略改进，直接收敛到  $V^*$ 
    nS, nA = env.observation_space.n, env.action_space.n
    V = np.zeros(nS) # 初始化  $V^*(s) = 0$ 

    while True:
        delta = 0

        # 贝尔曼最优方程：  $V^*(s) = \max_a \sum_{s'} P(s'|s,a) [R + \gamma V^*(s')]$ 
        for s in range(nS):
            q = np.zeros(nA) # 存储每个动作的 Q 值
            for a in range(nA):
                for (p, s_next, r, done) in env.unwrapped.P[s][a]:
                    q[a] += p * (r + gamma * V[s_next] * (1 - done))

            v_new = q.max() # 取 max：与策略评估的核心区别
            delta = max(delta, abs(v_new - V[s]))
            V[s] = v_new

        if delta < theta:
            break
    return V #  $V^*$  可通过一次策略改进得到  $\pi^*$ 
```

3.5 DP 的局限与现代意义

DP 完美解决了“已知 MDP 时如何求最优策略”的问题。但现实骨感——DP 在实际问题中几乎永远用不上。原因有三个：

- **问题 1：需要知道 P 和 R。** 现实中绝大多数环境不提供 P 和 R。例如自动驾驶，“前方有行人时减速 0.5g 会发生什么”无法精确写出公式。
 - **问题 2：状态空间太大。** DP 需要遍历所有状态，每个状态独立维护一个 V 值。但 Atari 游戏有 2^{1280} 个像素状态，围棋有 10^{170} 个棋盘状态——表格根本存不下。
 - **问题 3：维度灾难。** DP 要遍历所有 (s,a) 对，复杂度 $O(|S|^2|A|)$ ，状态空间稍大就爆炸。
- 这三个问题分别引出了 RL 的三条主线：

- **下一章的蒙特卡洛方法解决第一个问题：**不需要 P 和 R，用样本（经验）估计价值。这是从“模型已知”到“模型未知”的关键一步。
- **后续章节的函数逼近（DQN 等）解决第二个问题：**用神经网络替代表格，让 V/Q 可以泛化到未见过的状态。
- **采样方法（共同）解决第三个问题：**不遍历所有 (s,a)，只采样访问到的。

☒ 具体计算实例：2 状态 MDP 上的策略迭代收敛过程

沿用第二章的 2 状态 MDP (A、B 各有"留/走")， $\gamma=0.9$ 。展示策略迭代如何从随机策略收敛到最优。

初始策略 π_0 : 均匀随机 $\pi(a|s)=0.5$

第 1 轮: 策略评估 (参考 Ch2 结果)

$$V_1(A) \approx 7.25, V_1(B) \approx 7.75$$

第 1 轮: 策略改进 (贪心选 $\max Q$)

$$Q(A, \text{留}) = 1 + 0.9 \times 7.25 = 7.525; Q(A, \text{走}) = 0 + 0.9 \times 7.75 = 6.975$$

$$\rightarrow \pi_1(A) = \text{留} \quad (\text{因 } 7.525 > 6.975)$$

$$Q(B, \text{留}) = 2 + 0.9 \times 7.75 = 8.975; Q(B, \text{走}) = 0 + 0.9 \times 7.25 = 6.525$$

$$\rightarrow \pi_1(B) = \text{留} \quad (\text{因 } 8.975 > 6.525)$$

第 2 轮: 策略评估 (在新策略 π_1 下)

$$V_2(A) = 1 + 0.9 \cdot V_2(A) \rightarrow V_2(A) = 10$$

$$V_2(B) = 2 + 0.9 \cdot V_2(B) \rightarrow V_2(B) = 20$$

第 2 轮: 策略改进

$$Q(A, \text{留}) = 1 + 0.9 \times 10 = 10; Q(A, \text{走}) = 0 + 0.9 \times 20 = 18$$

$$\rightarrow \pi_2(A) = \text{走} \quad (\text{变化! } 18 > 10)$$

$$Q(B, \text{留}) = 2 + 0.9 \times 20 = 20; Q(B, \text{走}) = 0 + 0.9 \times 10 = 9$$

$$\rightarrow \pi_2(B) = \text{留} \quad (\text{不变})$$

第 3 轮: 策略评估

$$V_3(A) = 0.9 \cdot V_3(B) = 0.9 \times 20 = 18$$

$$V_3(B) = 2 + 0.9 \cdot V_3(B) \rightarrow V_3(B) = 20$$

第 3 轮: 策略改进

$$Q(A, \text{走}) = 0.9 \times 20 = 18 > Q(A, \text{留}) = 1 + 0.9 \times 18 = 17.2, \pi_3(A) = \text{走} \quad (\text{不变})$$

$$Q(B, \text{留}) = 2 + 0.9 \times 20 = 20 > Q(B, \text{走}) = 0.9 \times 18 = 16.2, \pi_3(B) = \text{留} \quad (\text{不变})$$

收敛! $\pi_3 = \pi_2$, 策略迭代结束。

最终: $V^*(A)=18, V^*(B)=20, \pi^*(A)=\text{走}, \pi^*(B)=\text{留}$ (与 Ch2 直接求解一致 $\checkmark$)

对比值迭代: 值迭代直接对 V^* 迭代, 从 $V_0=0$ 开始: $V_1(A)=\max(1, 0)=1, V_1(B)=\max(0, 2)=2$; $V_2(A)=\max(1+0.9, 0.9 \times 2)=1.9, V_2(B)=\max(0.9, 2+0.9 \times 2)=3.8$; 约 30 步后收敛到 $V^*(A)=18, V^*(B)=20$ 。

虽然 DP 在现实中很少直接使用，但它的思想——“贝尔曼备份”——是所有现代 RL 算法的核心。理解了 DP，再看后续算法就豁然开朗——它们都在解决同一个问题：如何在不知道 MDP 的情况下做 DP 能做的事。

[技巧] 运行 `ch03_dp.py` 后可以看到，策略迭代和值迭代在 FrozenLake 上都能达到 ~75% 的成功率。剩下的失败主要是冰面湿滑导致的随机性——即使最优策略也无法保证 100% 成功。

第四章 蒙特卡洛方法

第三章动态规划需要**完全已知 MDP**——也就是 P 和 R 都要写出来。但现实中我们几乎永远不知道 P 和 R：你不知道自动驾驶中“前方有行人时减速 0.5g 会发生什么”，也不知道下棋时“走这一步后对手会有什么反应的概率分布”。

蒙特卡洛 (Monte Carlo, MC) 方法的核心洞察是：**不需要知道 P 和 R，用样本（经验）来估计价值函数**。这是从“模型已知”到“模型未知”的关键一步——也标志着我们从 DP 走向了真正的 RL。

MC 方法的代价是：它要求环境是“分回合”的 (episodic)，且必须等回合结束才能更新——这两个限制将在下一章 TD 方法中被打破。本章我们先享受 MC 的简洁与优雅。

4.1 MC 预测

MC 预测 (Prediction) 要解决的问题是：给定策略 π ，估计 V^π 或 Q^π 。我们对比 DP 的做法，看看 MC 如何“绕过 P 和 R”。

DP 的做法：用 P 和 R 来“算” $V^\pi(s)$ 。回顾第三章策略评估的更新规则：

$$V^\pi(s) = \sum_a \pi(a|s) \cdot \sum_{s'} P(s'|s,a) \cdot [R(s,a,s') + \gamma \cdot V^\pi(s')]$$

这个公式里 P 和 R 是必需的——没有它们就算不出来。但 P 和 R 正是 DP 在现实中不可用的原因。

MC 的做法：不需要 P 和 R！它直接跑很多回合，记录每个状态的回报 G_t ，然后**取平均**就是 $V(s)$ 。为什么这样做有效？回到第二章 V 的定义：

$$V^\pi(s) = E_\pi[G_t \mid s_t = s]$$

$V^\pi(s)$ 是 G_t 的**期望**，而“样本平均”是期望的**无偏估计**——这就是 MC 的全部理论基础。说人话：从状态 s 出发跑很多次，把每次的回报加起来除以次数，平均下来就接近 $V^\pi(s)$ 。

一步步写出 MC 预测算法：

- 1. 跑 N 个完整回合（按策略 π 选动作）
- 2. 在每个回合中，记录每次经过状态 s 时的回报 G_t
- 3. 把所有回合中 s 出现时的 G_t 收集起来
- 4. $V(s) \approx (1/N) \sum G_t$ ，即样本平均

这就是 MC 预测的完整流程。注意到我们从来没有"用"过 P 或 R ——我们只是观察环境怎么响应（这天然就包含了 P 和 R 的影响），然后取平均。 P 和 R 隐式地"藏"在环境里，我们不需要知道它们的具体形式。

一个细节：在同一个回合里，状态 s 可能出现多次。如何处理？这引出两种变体：

- **首次访问 MC (First-visit MC)**：每个回合中只对 s 的**第一次**出现计算 G_t 并平均。
- **每次访问 MC (Every-visit MC)**：对 s 的**每次**出现都计算 G_t 并平均。

两者在无限样本下都收敛到 $V^\pi(s)$ ，但首次访问在数学上更易分析（每次访问的样本间不独立），每次访问在实践中更数据高效。两者的差异在大多数实际问题中可忽略。

4.2 MC 控制

MC 预测给了我们 $V(s)$ ，但要改进策略，我们还需要 $Q(s,a)$ 。这立刻引出一个问题：**DP 用 $Q^\pi(s,a) = \sum_s' P(s'|s,a)[R + \gamma V(s')]$ 来从 V 算 Q ，但 MC 不知道 P ！**

MC 的解法：直接估计 $Q(s,a)$ 而不是 $V(s)$ 。类比 MC 预测——只是把"状态 s 出现时的回报"换成" (s,a) 出现时的回报"。跑回合时记录每次执行 (s,a) 后的 G_t ，取平均就是 $Q(s,a)$ 。

但是这里有一个新的麻烦：**如果策略 π 是确定性的，某些 (s,a) 对可能永远不会被访问到**，这样它们的 Q 永远估计不出来，也就无法改进。这就是 MC 控制必须解决的"探索"问题。

4.2.1 探索策略： ϵ -greedy

最简单也最常用的探索策略是 **ϵ -greedy**：以 $1-\epsilon$ 概率选择贪心动作 $\arg\max_a Q(s,a)$ ，以 ϵ 概率随机选择动作。这保证所有动作都有非零被选概率，从而确保所有 (s,a) 对被持续访问——这是 MC 控制收敛的前提。 ϵ 通常取 0.1，也可以从 1.0 衰减到 0.05（前期多探索，后期多利用）。

4.2.2 on-policy MC 控制算法

完整的 on-policy MC 控制流程如下：

- 1. 初始化 $Q(s,a)$ 任意值， ϵ -soft 策略 π
- 2. 循环：生成一个回合（按 π ）
- 3. 对回合中每个 (s,a) 首次出现，计算 G （从该时刻到回合结束的回报）
- 4. $Q(s,a) \leftarrow Q(s,a) + \alpha(G - Q(s,a))$ （增量平均）
- 5. $\pi(s) \leftarrow \epsilon\text{-greedy}(Q(s,\cdot))$

步骤 4 用了"增量平均"技巧——不需要保存所有历史 G ，只需维护一个"当前估计 Q "，每次用 $Q \leftarrow Q + \alpha(G - Q)$ 更新。 α 是学习率（如 0.1），控制新样本的权重。当 $\alpha = 1/N$ 时退化为简单平均； α 较小时让旧估计平滑过渡。

4.3 Blackjack 实战

Blackjack（21 点）是 MC 的经典教学环境：玩家根据自己手牌总和与庄家明牌决定"要牌"或"停牌"。该环境是分回合的，回报只在回合末确定（+1 赢 / 0 平 / -1 输），非常适合 MC——因为中间状态没有有意义的即时奖励，DP 也无法直接用（不知道 P ）。

配套代码 `ch04_mc_control.py` 实现了完整的 MC 控制，50000 回合训练后，玩家胜率约 41-43%，已经接近理论最优（庄家有先手优势，玩家最优胜率约 43.5%）。观察学到的策略可以发现一些符合直觉的规律：

- 当玩家点数 ≥ 18 时总是停牌
- 当玩家点数 ≤ 11 时总是要牌（不会爆）
- 中间区域根据庄家明牌调整：庄家明牌大时更激进，庄家明牌小时更保守

4.4 MC 的优缺点

☒ 具体计算实例：MC 回报 G_t 计算与首次访问更新

考虑一个 3 步回合， $\gamma=0.9$ ：

轨迹： $(s_1, a_1, r=0) \rightarrow (s_2, a_2, r=0) \rightarrow (s_3, a_3, r=+10) \rightarrow \text{终止}$

第 1 步：从后向前计算回报 G_t

$G_3 = r_3 = 10$ （最后一步，无未来）

$G_2 = r_2 + \gamma \cdot G_3 = 0 + 0.9 \times 10 = 9$

$G_1 = r_1 + \gamma \cdot G_2 = 0 + 0.9 \times 9 = 8.1$

第 2 步：首次访问 MC 更新 Q （假设初始 $Q(s,a)=0$, $\alpha=0.5$ ）

$Q(s_1, a_1) \leftarrow Q(s_1, a_1) + \alpha(G_1 - Q(s_1, a_1)) = 0 + 0.5 \times (8.1 - 0) = 4.05$

$Q(s_2, a_2) \leftarrow 0 + 0.5 \times (9 - 0) = 4.5$

$Q(s_3, a_3) \leftarrow 0 + 0.5 \times (10 - 0) = 5.0$

第 3 步：若 s_1 在回合中再次出现（如 $s_1 \rightarrow s_2 \rightarrow s_1 \rightarrow s_3$ ）

首次访问：只用第一次出现的 G 更新（忽略第二次）

每次访问：两次出现都更新 $Q(s_1, a)$ ，平均两个 G 值

启示：MC 把"延迟奖励"反向传播到早期状态——虽然 s_1 当时得 0 分，但因后续得到 +10， s_1 的价值被"提升"到 4.05。这就是 RL 的核心：信用分配。

总结 MC 的优缺点，作为下一章 TD 方法的引子：

- **优点 1：无模型（model-free）。**不需要 P 和 R ，只需环境能交互即可。这把 RL 从 DP 的"理论玩具"变成了"实用工具"。
- **优点 2：无偏。**样本平均是期望的无偏估计——这意味着 MC 估计的 Q 在极限下严格收敛到真值，不会有系统性偏差。
- **缺点 1：必须等回合结束才能更新。**如果回合很长（甚至无限），MC 完全无法工作——这让它不适用于连续任务（如机器人持续控制）。

- **缺点 2：方差大。** G_t 是整条轨迹的累积和，单次回合的 G_t 可能波动很大，导致估计不稳定。
- **缺点 3：样本效率低。** on-policy MC 每个样本只用一次，且必须用当前策略产生的数据，无法复用旧策略的数据。

下一章的 TD（时序差分）方法解决前两个问题：它不需要等回合结束——只需一步就能更新（用单步估计 $r + \gamma V(s')$ 替代完整回报 G_t ），同时因为是单步估计，方差远小于 MC 的整段回报。TD 结合了 DP 的“单步更新”和 MC 的“无模型”特性，是现代 RL 的真正基石。

第五章 时序差分学习：SARSA 与 Q-Learning

第四章的蒙特卡洛（MC）方法解决了“不知道 MDP 模型”的问题——不需要转移概率 $P(s'|s,a)$ 与奖励函数 $R(s,a)$ ，只用真实样本就能估计价值。但 MC 留下了两个尚未解决的痛点：

- **痛点 1：必须等回合结束才能更新。** MC 用完整回报 $G_t = r_{t+1} + \gamma r_{t+2} + \dots + \gamma^{T-t-1} r_T$ ，其中 T 是回合终止时刻。对开车、机器人控制这类**连续任务**（没有“回合结束”），MC 根本无法工作。
- **痛点 2：方差大。** G_t 是从 t 到终止的完整累积奖励，每一步的随机性都被累乘进来，单个样本的 G_t 可能从 -100 到 $+1000$ 剧烈波动，导致梯度估计噪声极大、收敛极慢。

本章介绍的**时序差分（Temporal Difference, TD）学习**同时解决这两个问题：它**每一步就能更新**（无需等回合结束），并用**单步估计替代完整回报**（方差更小）。TD 是 MC 与 DP 的折中——既保留 MC 的“无模型”特性，又获得 DP 的“单步更新”优势。本章将依次推导 TD(0)、SARSA、Q-Learning、期望 SARSA 与 TD(λ) 五种核心算法。

5.1 TD(0) 预测

我们先解决最基础的问题：**给定策略 π ，如何估计 $V^\pi(s)$ ？** 回顾 MC 的更新公式：

$$V(s_t) \leftarrow V(s_t) + \alpha [G_t - V(s_t)]$$

这里 G_t 是从 t 时刻开始的完整回合回报。TD(0) 的**核心洞察**是：用“**一步真实奖励 + 一步估计值**”替代 G_t ：

$$G_t \approx r_{t+1} + \gamma V(s_{t+1})$$

为什么这个替代合理？因为根据贝尔曼方程 $V^\pi(s_t) = E[r_{t+1} + \gamma V^\pi(s_{t+1})]$ ，所以“**一步真实奖励 + 下一步的当前估计**”恰好是 $V(s_t)$ 的**无偏估计**（在 $V(s_{t+1})$ 准确的前提下）。把它代入 MC 更新公式，分两步得到 TD(0) 的更新规则：

第 1 步：把 G_t 替换为一步估计

$$V(s_t) \leftarrow V(s_t) + \alpha [r_{t+1} + \gamma V(s_{t+1}) - V(s_t)]$$

第 2 步：定义两个关键量

$$\text{TD 目标: } r_{t+1} + \gamma V(s_{t+1})$$

$$\text{TD 误差 } \delta_t: r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$$

于是更新公式可简记为 $V(s_t) \leftarrow V(s_t) + \alpha \delta_t$ 。关键性质：TD 用"一步真实 (r_{t+1}) + 一步估计 ($\gamma V(s_{t+1})$)"，而非 MC 的"完整真实回报"。这种"用估计更新估计"的方式叫**自举 (bootstrap)**。

偏差-方差权衡是理解 TD 的核心。三种方法对比：

- **MC**：用完整真实回报 G_t ——无偏，但方差大（累积 T 步随机性）
- **DP**：用一步估计 $r + \gamma V(s')$ ——方差小（只 1 步随机性），但需要 MDP 模型且有偏
- **TD**：用"一步真实 + 一步估计"——偏差与方差都小，且无需 MDP 模型

TD 的妙处在于：它不需要等回合结束（每步就能更新），方差也比 MC 小得多（只引入 1 步随机性）。代价是引入了"自举偏差"—— $V(s_{t+1})$ 是估计值而非真值，但理论上 TD(0) 在表格情形下仍收敛到 V^π 。

5.2 SARSA: on-policy TD 控制

5.1 解决了**策略评估**（估计 V ）。但**控制**（找最优策略）需要 $Q(s,a)$ ，而不是 $V(s)$ 。把 TD(0) 从 V 推广到 Q ，最自然的方式是：把状态值 V 换成动作值 Q ，把下一个状态 s' 换成"下一个状态-动作对 (s', a') "。分三步推导：

第 1 步：TD(0) 在 V 上的更新

$$V(s_t) \leftarrow V(s_t) + \alpha [r_{t+1} + \gamma V(s_{t+1}) - V(s_t)]$$

第 2 步：把 V 替换为 Q ，注意 Q 需要动作参数

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$$

第 3 步：明确 a_{t+1} 的来源

这里 a_{t+1} 不是任意的——它是用**当前策略 π** 在 s_{t+1} 处**实际采样的动作**（通常 ϵ -greedy）。因为更新用的样本 $(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$ 与生成数据的策略相同，所以 SARSA 是**on-policy**（**同策略**）算法。

名字 SARSA 来源于更新元组 **(S, A, R, S', A')**——这 5 个量构成一个完整转移，正是算法名的首字母。其完整更新规则为：

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma Q(s',a') - Q(s,a)]$$

on-policy 的含义：SARSA 学到的是"考虑探索的策略价值"。在悬崖边，SARSA 知道下一步 ϵ 探索可能让它掉下去，所以会主动远离悬崖——这是**保守策略**。


```

# SARSA 核心循环 (on-policy)
# 特点: 用实际采样的 next_action 更新 Q
state, _ = env.reset()
action = epsilon_greedy(Q, state, nA, epsilon) # 初始动作

while not done:
    # 执行动作, 获取转移
    next_state, reward, terminated, truncated, _ = env.step(action)
    # 重要: 在更新前先采样 next_action (on-policy 关键)
    next_action = epsilon_greedy(Q, next_state, nA, epsilon)

    # SARSA 更新:  $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma Q(s',a') - Q(s,a)]$ 
    # (1 - terminated): 终止状态不 bootstrap
    td_target = reward + gamma * Q[next_state, next_action] * (1 - terminated)
    Q[state, action] += alpha * (td_target - Q[state, action])

    # 同步推进 state/action (SARSA 名字来源: S,A,R,S',A')
    state, action = next_state, next_action

```

5.3 Q-Learning: off-policy TD 控制

SARSA 解决了 TD 控制问题, 但它有一个新问题: 用实际采样的 a_{t+1} 更新, 方差大且易受坏运气影响。如果 a_{t+1} 碰巧是个坏动作 (如悬崖边的"掉下去"), $Q(s,a)$ 就会被这个坏样本拉低——即使你策略本身在 s' 处大多数时候选了好动作。

Q-Learning 的核心思想: 不用实际采样的 a_{t+1} , 而用**最优**的 a' 。分两步推导:

第 1 步: SARSA 用实际采样的 a'

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma Q(s',a') - Q(s,a)]$$

第 2 步: 把 $Q(s',a')$ 替换为 $\max_{a'} Q(s',a')$

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

这一替换带来本质变化:

- **行为策略** (用来与环境交互、收集数据) 可以是 ϵ -greedy——保证探索;
- **目标策略** (用来定义 Q 的更新目标) 是 greedy——直接学最优。

因为行为策略 $\neq$ 目标策略, Q-Learning 是**off-policy (异策略)**算法。它的**好处**是数据可重复使用——来自任意行为策略的样本都能更新同一个 Q 。在悬崖边, Q-Learning 假设"未来不再探索", 所以学最优贴边路径——但若实际执行 ϵ -greedy, 会因偶尔探索而掉崖。

```
# Q-Learning 核心循环 (off-policy)
# 特点: 用  $\max_a Q(s', a')$  更新, 与采样动作无关
state, _ = env.reset()

while not done:
    # 行为策略:  $\epsilon$ -greedy (用于探索)
    action = epsilon_greedy(Q, state, nA, epsilon)
    next_state, reward, terminated, truncated, _ = env.step(action)

    # 目标策略: greedy (用  $\max_a Q$  计算 TD 目标)
    # 关键区别: 不需要采样 next_action!
    next_q = Q[next_state].max() * (1 - terminated)
    td_target = reward + gamma * next_q
    Q[state, action] += alpha * (td_target - Q[state, action])

    state = next_state
```

5.4 经典对比: CliffWalking

Sutton & Barto 教科书中的 CliffWalking 是对比 SARSA 与 Q-Learning 的最佳例子。环境是一个 4×12 网格, 左下是起点, 右下是目标, 最下面一行是"悬崖"——掉下去得 -100 并回到起点。普通移动每步 -1, 所以最短路径(贴边走)总回报 -13; 保守路径(绕开悬崖)约 -25 至 -50。

运行 `ch05_td_sarsa_q.py` 可以观察到经典现象:

- **SARSA** 学到保守路径——绕开悬崖走, 在线性能更稳定 (约 -25)。
- **Q-Learning** 学到最优路径——贴边走, 但训练期因 ϵ 探索常掉悬崖 (约 -50)。

这个对比揭示了 on-policy 与 off-policy 的本质差异: on-policy 学"我会怎么走" (考虑探索), off-policy 学"最优怎么走" (不考虑探索)。在实践中, off-policy 数据效率更高 (数据可重复使用), 但训练可能不稳定。

5.5 期望 SARSA 与算法选择

☒ 具体计算实例: SARSA vs Q-Learning 更新对比

设当前 Q 表 (部分), $\alpha=0.5, \gamma=0.9, \epsilon=0$ (贪心):

$$Q(s, a_1) = 5, Q(s, a_2) = 10, Q(s', a_1) = 2, Q(s', a_2) = 8$$

智能体在 s 执行 a_2 , 得 $r=1$, 转到 s' 。在 s' 处:

SARSA 更新 (假设在 s' 实际采样到 a_1):

$$\text{TD target} = r + \gamma \cdot Q(s', a_1) = 1 + 0.9 \times 2 = 2.8$$

$$Q(s, a_2) \leftarrow 10 + 0.5 \times (2.8 - 10) = 10 - 3.6 = 6.4$$

Q-Learning 更新 (用 $\max Q(s', \cdot)$):

$$\text{TD target} = r + \gamma \cdot \max Q(s', \cdot) = 1 + 0.9 \times 8 = 8.2$$

$$Q(s, a_2) \leftarrow 10 + 0.5 \times (8.2 - 10) = 10 - 0.9 = 9.1$$

差异分析：

SARSA 大幅降低 $Q(s, a_2)$ (10→6.4)，因实际采样到低价值的 a_1

Q-Learning 假设未来最优，贴边走。

Expected SARSA (折中)：

$$\text{TD target} = r + \gamma \cdot E[Q(s', \cdot)] = 1 + 0.9 \times (0.5 \times 2 + 0.5 \times 8) = 1 + 0.9 \times 5 = 5.5$$

$$Q(s, a_2) \leftarrow 10 + 0.5 \times (5.5 - 10) = 7.75 \text{ (介于两者之间)}$$

上面的数值例子揭示了 SARSA 与 Q-Learning 各自的痛点：**SARSA** 用采样的 a' ，方差大（运气差时大幅偏离）；**Q-Learning** 用 $\max_{a'}$ ，会**过估**（max 操作放大正向噪声）。**期望 SARSA** (Expected SARSA) 是两者的折中：用期望替代采样，用平均替代 max：

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \sum_{a'} \pi(a'|s') Q(s', a') - Q(s, a)]$$

把所有可能的 a' 按策略概率 $\pi(a'|s')$ 加权求和——这**消除了 a' 的采样随机性**（不再受运气影响），又**不会过估**（用平均而非最大）。由于消除了 a' 的采样随机性，期望 SARSA 比 SARSA 方差更小；而用期望替代 max 又避免了 Q-Learning 的过估问题。当动作空间小时（如 4 个），计算期望的代价可忽略；动作空间大时则成本较高。

[技巧] 选择建议：若环境回报方差大、训练不稳定 → 优先 SARSA 或 Expected SARSA；若数据有限、需要 off-policy 重用 → 优先 Q-Learning（及后续的 DQN 系列）。

5.6 TD(λ): 在 MC 与 TD 之间插值

TD(0) 只用 1 步回报 ($r + \gamma V(s')$)，偏差大但方差小；MC 用 T 步完整回报，无偏但方差大。**能否在 1 步与 T 步之间插值？**——这就是 n-step TD：

第 1 步：定义 n-step return（前 n 步真实 + 第 n 步的估计）

$$G_t^{(n)} = r_{t+1} + \gamma \cdot r_{t+2} + \dots + \gamma^{n-1} \cdot r_{t+n} + \gamma^n \cdot V(s_{t+n})$$

第 2 步：观察两个极端

- $n=1$ ：退化为 TD(0)（只有 1 步真实，剩下用 V 估计）
- $n=T$ ：退化为 MC（全部用真实回报，无自举）

第 3 步：TD(λ) 用几何加权平均所有 n-step return

$$G_t^\lambda = (1-\lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}$$

当 $\lambda=0$ 时退化为 TD(0)（只有 $n=1$ 项权重为 1）， $\lambda=1$ 时退化为 MC（所有 n 等权加权，等于完整回报）。实践中 $n=3$ 到 $n=5$ 的多步回报（或 $\lambda=0.95$ 附近的 TD(λ)）能在偏差与方差间取得良好平衡。现代 RL 算法如 Rainbow DQN 中的 N-step Returns 组件就是这种思路。

本章介绍的 TD 方法（SARSA、Q-Learning、Expected SARSA、TD(λ)）都用**表格**存储每个 (s,a) 的 Q 值。但当状态空间变大时，表格变得不现实——Atari 游戏的像素状态是 $84 \times 84 \times 4 \approx 10^5$ 维，围棋棋盘有 $3^{361} \approx 10^{170}$ 个状态，表格根本存不下，更不用说"遍历每个状态学习"了。下一章将引入**函数逼近**——用神经网络 $Q_\theta(s,a)$ 替代表格，让 RL 第一次能处理高维状态，这正是 DQN 的核心思想。

第六章 函数逼近与 DQN

第五章的 Q-Learning 用**表格**存储每个 (s,a) 的 Q 值——表格法假设状态空间有限。但 Atari 游戏的状态是 $84 \times 84 \times 4$ 的像素（约 10^5 维），围棋有 $3^{361} \approx 10^{170}$ 个状态，**表格根本存不下，更不可能遍历每个状态学习**。

函数逼近用参数化函数 $Q_\theta(s,a)$ 替代表格——给定任意状态 s，神经网络输出一个 Q 值，无需为每个状态分配独立存储。这看似简单，但 2013 年前没人能让神经网络 Q 函数稳定训练。DeepMind 2013 年的 DQN（Deep Q-Network）通过三大创新首次实现这一点，在 49 个 Atari 游戏上达到人类水平——这是深度 RL 的里程碑。

6.1 函数逼近基础

函数逼近的核心是用参数 θ 近似价值函数： $V_\theta(s) \approx V^\pi(s)$ 或 $Q_\theta(s,a) \approx Q^\pi(s,a)$ 。 θ 可以是线性特征模型的权重，也可以是深度神经网络的参数。学习目标是**最小化均方误差**：

$$J(\theta) = E[(G_t - Q_\theta(s_t, a_t))^2]$$

梯度下降更新规则（分两步推导）：

第 1 步：对 θ 求梯度

$$\nabla_\theta J(\theta) = E[2 (G_t - Q_\theta(s,a)) \cdot (-\nabla_\theta Q_\theta(s,a))]$$

第 2 步：去掉常数因子 2 与符号，得到 SGD 更新

$$\theta \leftarrow \theta + \alpha (G_t - Q_\theta(s,a)) \nabla_\theta Q_\theta(s,a)$$

若用 TD 目标 $r + \gamma Q(s',a')$ 替代 G_t ，得到"半梯度 TD"算法——之所以叫**半梯度**，是因为目标值 $Q(s',a')$ 也依赖 θ ，但更新时**忽略了这一项的梯度**。在 on-policy 下这种方法收敛，但 off-policy 下可能发散——这正是 DQN 引入目标网络的原因（见 6.2）。

问题：直接用 SGD 训练神经网络 Q 函数**不稳定**。2013 年之前，所有尝试都失败——网络要么不收敛，要么训练中突然崩溃。根本原因有 3 个，DQN 用 3 个对应的创新逐一解决。

6.2 DQN 的三大创新

DQN 的三大创新——对应于"为什么直接 SGD 训练不稳定"的 3 个原因：

6.2.1 经验回放（Experience Replay）—— 解决数据相关性

问题 1: 神经网络假设输入是 i.i.d. (独立同分布) 样本, 但 RL 中相邻状态高度相关 (同一回合的连续帧几乎相同), 违反 i.i.d. 假设, 导致 SGD 偏差严重。

解决 1: 把每一步 (s, a, r, s', done) 存入**回放缓冲区**, 训练时**随机采样 mini-batch**。随机采样打破了时间相关性——一个 batch 内的样本可能来自不同回合、不同时期, 近似 i.i.d.。这一招一举三得:

- 打破数据时间相关性 (近似 i.i.d.)
- 提高数据利用率 (每个样本可被多次学习)
- 稳定训练 (避免最近经验过拟合)

回放缓冲区通常大小为 10^4 – 10^6 , 采用 FIFO 策略。

6.2.2 目标网络 (Target Network) —— 解决"追逐自己的尾巴"

问题 2: TD 目标 $r + \gamma Q(s', a')$ 中的 $Q(s', a')$ 依赖 θ , 每次更新 θ 后目标值立刻变化——相当于"在追一个移动的靶子", 训练严重不稳定 (这叫**追逐自己的尾巴**问题)。

解决 2: 维护**两个网络**——在线网络 Q_θ (持续更新) 与目标网络 $Q_{\theta'}$ (计算 TD 目标)。每 N 步 (通常 $N=10$ – 1000) 把 θ 同步到 θ' 。两次同步之间, 目标值是固定的, 相当于"靶子不动"。

第 1 步: 写朴素 TD 损失 (无目标网络)

$$L(\theta) = E[(r + \gamma \max_a Q_\theta(s', a') - Q_\theta(s, a))^2]$$

第 2 步: 把目标值里的 θ 换成 θ' (停止梯度), 得到 DQN 损失

$$L(\theta) = E[(r + \gamma \max_a Q_{\theta'}(s', a') - Q_\theta(s, a))^2]$$

关键点: θ' 在两次同步之间是**常数** (停止梯度), 所以目标值不随 θ 变化, 训练稳定下来。

6.2.3 ϵ -greedy 探索 + 帧堆叠

问题 3: 神经网络初始化时倾向于输出确定性策略, 导致探索不足。

解决 3: 用 ϵ -greedy 探索 (ϵ 从 1.0 衰减到 0.05), 并采用 4 帧堆叠作为状态 (让网络感知速度)。这些工程细节看似简单, 但对最终性能至关重要——许多复现失败都是因忽略了这些细节。

6.3 DQN 完整算法

下面是 DQN 的伪代码, 配套代码 `ch06_dqn.py` 是完整 PyTorch 实现。

```

# DQN 主循环: 经验回放 + 目标网络
for episode in range(num_episodes):
    state = env.reset()
    for t in range(max_steps):
        # ε-greedy: 以 ε 概率随机探索, 否则选 argmax Q
        action = agent.act(state)
        # 执行动作, 获取转移
        next_state, reward, done, _ = env.step(action)
        # 存入经验回放缓冲区 (打破数据时间相关性)
        buffer.push(state, action, reward, next_state, done)
        state = next_state
    # 从 buffer 随机采样 batch 训练
    agent.learn()
    # 每 target_sync 个 episode 同步一次目标网络
    # (避免目标值跟随 θ 立即变化导致不稳定)
    if episode % target_sync == 0:
        agent.sync_target()

```

其中 `agent.learn()` 的核心代码如下:

```

# DQN learn() 核心: TD 更新 + 目标网络
def learn(self):
    # 从经验回放缓冲区随机采样 batch
    batch = buffer.sample(64)
    s, a, r, s_next, done = batch

    # 1. 计算当前 Q(s,a) - 用 policy_net
    # gather(1, a): 按动作索引取出对应的 Q 值
    q_sa = policy_net(s).gather(1, a).squeeze()

    # 2. 计算 TD 目标: r + γ max_a' Q_target(s', a')
    with torch.no_grad(): # 不需梯度 (目标值是固定的)
        q_next = target_net(s_next).max(dim=1)[0] # max_a' Q(s', a')
        q_target = r + gamma * q_next * (1 - done) # (1-done): 终止不 bootstrap

    # 3. MSE 损失 + 反向传播
    loss = mse_loss(q_sa, q_target)
    optimizer.zero_grad()
    loss.backward()
    # 梯度裁剪: 防止 Q 值爆炸
    clip_grad_norm_(policy_net.parameters(), 10.0)
    optimizer.step()

```

6.4 训练技巧与超参数

☒ 具体计算实例: DQN 单步 TD 更新数值演示

设 CartPole 上采到一个转移 $(s, a, r, s', done)$, 参数 $\gamma=0.99$, $lr=1e-3$ 。

神经网络当前输出:

$Q_{\theta}(s) = [3.2, 5.8]$ (动作 0 价值 3.2, 动作 1 价值 5.8)

$Q_{\theta}(s') = [4.1, 6.5]$

智能体执行了 $a=1$ （动作 1），得 $r=1.0$ ，未终止 ($done=False$)。

第 1 步：取出当前 $Q(s,a)$

$$q_{sa} = Q_{\theta}(s)[1] = 5.8$$

第 2 步：计算 TD 目标（用 $target_net$ ，假设与 $policy_net$ 相同）

$$q_{next} = \max Q_{target}(s') = \max(4.1, 6.5) = 6.5$$

$$q_{target} = r + \gamma \cdot q_{next} \cdot (1 - done) = 1.0 + 0.99 \times 6.5 \times 1 = 7.435$$

第 3 步：计算 MSE 损失

$$loss = (q_{sa} - q_{target})^2 = (5.8 - 7.435)^2 = 2.673$$

第 4 步：反向传播更新 θ

$$\partial loss / \partial q_{sa} = 2 \times (5.8 - 7.435) = -3.27$$

经反向传播后， $Q_{\theta}(s)[1]$ 会被略微上调（朝 7.435 靠近）

若 $lr=1e-3$ ，单步更新后约 $Q_{\theta}(s)[1] \approx 5.8 + 1e-3 \times 3.27 \approx 5.803$

启示：DQN 每步只小幅调整 Q 值（约 0.003），但经过数千次更新后， Q 会逐步逼近真值。这就是为什么 RL 训练需要大量样本——单步信息量很小。

DQN 在 CartPole-v1 上的训练并不困难，但要达到 500 分稳定需要调参。下表是经过实践验证的推荐超参数：

超参数	推荐值	说明
学习率 lr	$1e-3$	Adam 优化器；过高训练不稳，过低收敛慢
折扣因子 γ	0.99	CartPole 几乎无随机性，可接近 1
回放缓冲区大小	10,000	CartPole 状态简单；Atari 需 $1e5-1e6$
批量大小	64	越大越稳但越慢；常用 32-256
目标网络同步	每 10 episodes	或用软更新 $\tau=0.005$
ϵ 衰减	$\exp(-steps/500)$	从 1.0 衰减到 0.05
梯度裁剪	10.0	防止 Q 值爆炸
奖励塑形	终止罚 -10	CartPole 经验技巧

表 6.1: DQN 在 CartPole-v1 上的推荐超参数

6.5 DQN 的局限

尽管 DQN 是巨大突破，但它仍留下 4 个明显问题，分别对应后续章节的解决方案：

- **问题 1: Q 值过估。**max 操作会放大正向噪声, 导致 Q 系统性高估真值 → **解决:** Double DQN (第十一章), 用目标网络选动作、在线网络估值, 解耦"选择"与"评估"。
- **问题 2: 无法处理连续动作。** $\max_a Q(s', a')$ 在离散空间是简单遍历, 但连续空间下 max 不可微、不可解析 → **解决:** 策略梯度 (第七章) / SAC (第十章), 直接参数化策略。
- **问题 3: 样本效率低。**经验回放均匀采样, 浪费了高价值样本 (罕见但信息量大) → **解决:** 优先经验回放 PER (第十一章), 按 TD 误差大小加权采样。
- **问题 4: 间接策略。**DQN 先学 Q^* , 再 $\arg\max$ 得到策略——这种"先评估后决策"的两步法在部分可观察环境、随机策略要求下受限 → **解决:** 策略梯度 (第七章), 直接参数化策略 $\pi_\theta(a|s)$ 。

问题 2 与问题 4 指向同一个新方向: 与其先学 Q 再 $\arg\max$, 不如**直接参数化策略** $\pi_\theta(a|s)$, 绕过 Q 函数这个中间人。这同时解决了连续动作 (不需要 max over a) 与间接策略 (直接优化策略本身) 两个问题——这就是下一章**策略梯度方法**的出发点。

第七章 策略梯度方法: REINFORCE

第六章末尾我们指出 DQN 有两大局限: **无法处理连续动作** (max 不可行)、**间接优化策略** (先学 Q 再 $\arg\max$)。本章的策略梯度方法直接参数化策略 $\pi_\theta(a|s)$ ——不再经过 Q 这个中间人, 绕开了这两个限制。

直接参数化策略的好处: (1) **连续动作:** $\pi_\theta(a|s)$ 可输出连续分布 (如高斯), 无需 max over a; (2) **随机策略:** $\pi_\theta(a|s)$ 自然表达随机策略, 在部分可观察环境 (如扑克) 有优势; (3) **直接优化:** 梯度直接作用于策略本身, 理论清晰。本章推导策略梯度定理, 并介绍最简单的策略梯度算法 REINFORCE。

7.1 策略目标函数

策略梯度直接优化策略参数 θ , 使其在某种意义上"更好"。常用目标函数有三种:

- **episodic 情形:** $J(\theta) = E_{\pi_\theta} [G_0]$ (起始状态价值)
- **continuing 情形:** $J(\theta) = \text{平均奖励}$ (每步期望回报)
- **discounted 情形:** $J(\theta) = E[\sum \gamma^t r_t]$

无论哪种, 目标都是**最大化 $J(\theta)$** 。策略梯度方法的核心问题是: **如何计算 $\nabla_\theta J(\theta)$** ? ——这正是策略梯度定理要回答的。难点在于: 状态分布 $\mu(s)$ 也依赖 θ (不同策略访问不同状态), 直接对 J 求导会出现 $\nabla_\theta \mu(s)$ 项, 难以计算。

7.2 策略梯度定理

策略梯度定理 (Policy Gradient Theorem) 是 RL 理论中最重要的结论之一。它告诉我们: **即使状态分布依赖 θ , $\nabla_\theta J(\theta)$ 仍可以写成不依赖 $\nabla_\theta \mu(s)$ 的简洁形式。**

第 1 步: 写出 $J(\theta)$ 的展开形式 (用状态分布 $\mu(s)$ 与策略 π_θ)

$$J(\theta) = E_{\pi_\theta} [G_0] = \sum_s \mu(s) \sum_a \pi_\theta(a|s) Q^{\pi_\theta}(s, a)$$

第 2 步：对 θ 求导，第一项 $\nabla_{\theta} \mu(s)$ 可以证明省略（详见 Sutton & Barto 13.5）

$$\nabla_{\theta} J(\theta) = \sum_s \mu(s) \sum_a \nabla_{\theta} \pi_{\theta}(a|s) \cdot Q^{\pi_{\theta}}(s,a)$$

第 3 步：用"对数导数技巧" $\nabla \pi = \pi \cdot \nabla \log \pi$ （这是关键一步，把概率梯度转为对数梯度）

$$\nabla_{\theta} J(\theta) = \sum_s \mu(s) \sum_a \pi_{\theta}(a|s) \cdot \nabla_{\theta} \log \pi_{\theta}(a|s) \cdot Q^{\pi_{\theta}}(s,a)$$

第 4 步：把求和改写为期望（ $\sum \pi \cdot = E_{\pi}$ ）

$$\nabla_{\theta} J(\theta) = E_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a|s) \cdot Q^{\pi_{\theta}}(s,a)]$$

关键概念解释：

- **得分函数（score function）：** $\nabla_{\theta} \log \pi_{\theta}(a|s)$ 告诉我们"朝哪个方向调 θ 能增大 $\pi(a|s)$ "——这是策略梯度的核心数学对象。

- **Q 作为权重：** $Q^{\pi_{\theta}}(s,a)$ 是"动作好坏"的评分——好动作 Q 大 $\rightarrow$ 增大概率；坏动作 Q 小 $\rightarrow$ 减小概率。

直观理解：策略梯度定理告诉我们——"在当前策略下采样 (s,a) ，按 $\nabla \log \pi \cdot Q$ 加权求和"就是 ∇J 。这正是"好动作多出现，坏动作少出现"的本质。它的一大优势是**无需 MDP 模型**——所有量都从样本估计。

7.3 REINFORCE 算法

策略梯度定理给出了 ∇J 的形式，但 $Q^{\pi_{\theta}}(s,a)$ 未知。**REINFORCE (Williams 1992)** 是最简单的策略梯度算法：用**实际回报 G_t** 替代 $Q^{\pi}(s,a)$ ——因为 $E[G_t | s_t, a_t] = Q^{\pi}(s,a)$ ，所以 G_t 是 Q 的无偏估计。

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \cdot G_t$$

算法流程：

- 1. 用当前策略 π_{θ} 生成一个完整回合
- 2. 对每个 (s_t, a_t) ，计算 G_t （从 t 到回合结束的回报）
- 3. 累加梯度 $\nabla \log \pi(a_t|s_t) \cdot G_t$
- 4. 回合结束后用累计梯度更新 θ

```

# REINFORCE 核心 PyTorch 实现 (with baseline)

# — 阶段 1: 采样一个完整回合 —
log_probs, rewards = [], []
for t in range(T):
    # 策略网络输出 logits (未归一化的对数概率)
    logits = policy(state)
    # 构造分类分布, 采样动作
    dist = Categorical(logits=logits)
    action = dist.sample()
    # 保存 log π(a|s) 用于后续梯度计算
    log_probs.append(dist.log_prob(action))
    state, reward, done, _ = env.step(action.item())
    rewards.append(reward)

# — 阶段 2: 计算回报 G_t (从后向前累积) —
returns = []
G = 0
for r in reversed(rewards):
    G = gamma * G + r # G_t = r_{t+1} + γ G_{t+1}
    returns.insert(0, G) # 插入到列表开头

# — 阶段 3: 策略梯度更新 —
# 目标: 最大化 ∑ log π(a_t|s_t) * G_t
# 取负号因为 PyTorch 默认做梯度下降
loss = -(torch.stack(log_probs) * torch.tensor(returns)).sum()
optimizer.zero_grad()
loss.backward()
optimizer.step()

```

7.4 减小方差：基线与优势

☒ 具体计算实例：REINFORCE 策略梯度数值计算

设 2 动作离散策略 $\pi_\theta(a|s) = \text{softmax}(\text{logits})$, 当前 logits = [1.0, 2.0], $\gamma=0.9$ 。

一个 3 步回合: $(s_1, a=1, r=0) \rightarrow (s_2, a=0, r=0) \rightarrow (s_3, a=1, r=+10) \rightarrow \text{终止}$

第 1 步：计算策略概率

$$\pi(a_0|s) = e^1/(e^1+e^2) = 0.269, \quad \pi(a_1|s) = e^2/(e^1+e^2) = 0.731$$

第 2 步：计算 $\log \pi(a|s)$

$$\log \pi(a_1|s) = \log(0.731) = -0.313 \quad (\text{动作 1 被采样})$$

第 3 步：计算回报 G_t (沿用 Ch4 结果)

$$G_1 = 8.1, G_2 = 9, G_3 = 10$$

第 4 步：计算策略梯度

$$\nabla_{\theta} J = \sum \nabla \log \pi(a_t | s_t) \cdot G_t$$

对 $\text{logits}[1]$ 的梯度（简化，假设三个状态策略相同）：

$$\nabla \log \pi(a_1 | s) / \partial \text{logits}[1] = 1 - \pi(a_1 | s) = 1 - 0.731 = 0.269$$

$$\text{总梯度} \approx 0.269 \times (G_1 + G_2 + G_3) = 0.269 \times 27.1 \approx 7.29$$

第 5 步：更新 θ ($\text{lr}=0.1$)

$$\text{logits}[1] \leftarrow \text{logits}[1] + \text{lr} \times 7.29 = 2.0 + 0.729 = 2.729$$

$$\text{更新后 } \pi(a_1 | s) = e^{2.729} / (e^1 + e^{2.729}) \approx 0.827 \quad (\text{动作 1 概率上升 } \checkmark)$$

启示：正回报 ($G>0$) 增大被采样动作概率，负回报 ($G<0$) 减小概率。这就是策略梯度的本质——“好动作多做，坏动作少做”。

上面的数值例子展现了 REINFORCE 的机制，但也暴露了它的**致命问题：方差极大**。单个回合的 G_t 可能从 -100 到 $+1000$ ，导致梯度估计噪声极大，训练极慢甚至不收敛。

减方差技巧：减去一个基线 $b(s)$

$$\nabla_{\theta} J(\theta) = E[\nabla \log \pi(a|s) \cdot (Q(s,a) - b(s))]$$

为什么减基线不影响期望？——一步推导：

第 1 步：把基线项单独写

$$E[\nabla \log \pi \cdot b(s)] = b(s) \cdot E_{a \sim \pi}[\nabla \log \pi(a|s)]$$

第 2 步：用 $\nabla \log \pi = \nabla \pi / \pi$ 化简

$$\begin{aligned} E_{a \sim \pi}[\nabla \log \pi] &= \sum_a \pi(a|s) \cdot \nabla \log \pi(a|s) = \sum_a \pi(a|s) \cdot \nabla \pi(a|s) / \pi(a|s) = \sum_a \nabla \pi(a|s) \\ &= \nabla_{\theta} \sum_a \pi(a|s) = \nabla_{\theta} 1 = 0 \end{aligned}$$

结论：基线项的期望恒为 0，所以减基线**不改变梯度期望**（无偏性不变），但**大幅减小方差**——因为 $b(s)$ 把“状态本身好坏”这个常数部分扣除了，剩下的 $Q - b$ 只反映“动作比平均好多少”。

最常用的 $b(s)$ 是 $V(s)$ ，此时 $Q(s,a) - V(s)$ 被称为**优势函数** $A(s,a) = Q(s,a) - V(s)$ 。直观上： $A > 0$ 表示该动作比平均好，应增加概率； $A < 0$ 表示比平均差，应减少概率。 $V(s)$ 可以用另一个网络（Critic）学习，这就引出了下一章的 Actor-Critic 框架。另一个常见技巧是**回报标准化**：将 G 标准化为 $(G - \text{mean}) / (\text{std} + \epsilon)$ ，消除不同回合间奖励尺度差异。

7.5 REINFORCE 的局限与改进方向

REINFORCE 留下 3 个明显问题，分别对应后续章节的解决方案：

· **问题 1：必须等回合结束才能更新**——因为 G_t 是完整回报。这对连续任务（开车、机器人）不适用。

- **问题 2：方差大**——即使加基线， G_t 仍包含从 t 到回合结束的全部随机性。
- **问题 3：样本效率低**——on-policy，每个样本只用一次就丢弃。

这 3 个问题分别由后续章节解决：**问题 1 与 2** → Actor-Critic（第八章）用 TD 误差 δ_t 替代 G_t ——每步就能更新（无需等回合），且方差大幅降低（只用 1 步随机性）；**问题 3** → PPO（第九章）用**重要性采样重复利用数据**——同一批数据训练多个 epoch，在不偏离 on-policy 安全性前提下大幅提升样本效率。

第八章 Actor-Critic 与 A2C/A3C

第七章末尾我们指出 REINFORCE 有两个核心问题：**必须等回合结束、方差大**。这两个问题的根源是同一个——使用完整回报 G_t 作为"动作好坏"的评分。**Actor-Critic (AC) 框架**用一个简单替换同时解决两个问题：用 TD 误差 δ_t 替代 G_t 。 δ_t 只需要一步转移就能计算，所以每步就能更新（不等回合结束）； δ_t 只含一步随机性，所以方差大幅降低。

8.1 AC 框架的核心思想

回顾 REINFORCE 的更新公式：

$$\theta \leftarrow \theta + \alpha \nabla \log \pi_{\theta}(a_t | s_t) \cdot G_t$$

这里 G_t 是"动作 a_t 有多好"的评分——但要等回合结束才知道，且方差大。**Actor-Critic 的洞察**：用 TD 误差 δ_t 替代 G_t 。分两步推导为什么这个替换合理：

第 1 步：写出 δ_t 的定义

$$\delta_t = r_{t+1} + \gamma V_{\phi}(s_{t+1}) - V_{\phi}(s_t)$$

第 2 步：对 δ_t 求条件期望（已知 s_t, a_t ）

$$E[\delta_t | s_t, a_t] = E[r_{t+1} + \gamma V(s_{t+1}) | s_t, a_t] - V(s_t) = Q^{\pi}(s_t, a_t) - V^{\pi}(s_t) = A^{\pi}(s_t, a_t)$$

结论： δ_t 是优势函数 $A(s_t, a_t)$ 的**无偏估计**，但方差远小于 G_t （只含一步随机性）。于是更新规则变为：

$$\begin{aligned} \theta &\leftarrow \theta + \alpha \nabla \log \pi_{\theta}(a_t | s_t) \cdot \delta_t \\ \phi &\leftarrow \phi + \beta \delta_t \nabla V_{\phi}(s_t) \end{aligned}$$

角色分工：

- **Actor（策略网络 π_{θ} ）**：用 δ_t 更新策略—— $\delta_t > 0$ 表示动作好 → 增大概率； $\delta_t < 0$ 表示动作差 → 减小概率。
- **Critic（价值网络 V_{ϕ} ）**：用 TD 误差更新 V ——这就是第五章的 TD(0) 预测， V 朝 TD 目标 $r + \gamma V(s')$ 靠近。

两个网络协同：Critic 评估状态好坏 → Actor 据此调策略 → 新策略产生新数据 → Critic 再评估。这种"评估-改进"循环正是策略迭代的思想，但每步都能执行（不再需要等回合结束）。与 REINFORCE 相比，AC 用"一步估计"替代"完整回报"，方差大幅降低，且支持在线学习。

8.2 A2C: 同步 Actor-Critic

A2C (Advantage Actor-Critic) 是 AC 的标准实现。关键细节:

- Actor 与 Critic 共享底层网络 (部分参数), 输出端分头 (policy head + value head)
- 用多步累积 (rollout) 代替单步 TD, 进一步降低方差
- 加入熵正则化 (entropy bonus) 鼓励探索
- 总损失 = actor_loss + 0.5 * critic_loss + 0.01 * entropy_loss

```
# A2C 总损失: actor + critic + entropy

# 1. Actor 损失: 最大化  $\log \pi(a|s) \times \text{优势}$ 
# 优势 = returns - values (TD 误差的近似)
# detach(): 优势作为常数, 不反传到 critic
actor_loss = -(log_probs * advantages.detach()).mean()

# 2. Critic 损失:  $V(s)$  应接近 returns
critic_loss = mse_loss(values, returns)

# 3. 熵正则: 鼓励策略保持随机性 (防止过早收敛)
# 取负号因为我们要最大化熵
entropy_loss = -dist.entropy().mean()

# 总损失 = actor + 0.5*critic + 0.01*entropy
# 系数 0.5/0.01 是经验值
total_loss = actor_loss + 0.5 * critic_loss + 0.01 * entropy_loss
```

熵正则化非常重要: 没有它, 策略会快速收敛到确定性策略而停止探索; 有它则保持一定随机性, 避免陷入局部最优。系数 0.01 是常用值。

8.3 A3C: 异步并行

A3C (Asynchronous Advantage Actor-Critic) 是 DeepMind 2016 年的工作, 与 A2C 的算法完全相同, 区别在于"异步并行":

- 多个 worker 进程同时与环境交互, 各自维护一份策略副本
- worker 定期将梯度推送到全局网络, 并从全局网络同步参数
- 不同 worker 经历不同状态, 相当于"内在探索"

异步并行的副作用是消除数据相关性 (不同 worker 的样本是 i.i.d. 的), 类似于 DQN 的经验回放, 但不需要缓冲区。A3C 在 Atari 上首次达到了与 DQN 相当的性能, 但训练速度快数倍。

8.4 GAE: 广义优势估计

☒ 具体计算实例: A2C 单步更新数值演示

设当前状态 s , Critic 输出 $V(s)=5.0$, 下一状态 s' 的 $V(s')=7.0$ 。

Actor 输出 logits=[0.5, 1.5], 采样到 $a=1$, 得 $r=1.0$, $done=False$, $\gamma=0.99$ 。

第 1 步：计算 TD 误差 δ (作为优势 A 的估计)

$$\delta = r + \gamma \cdot V(s') - V(s) = 1.0 + 0.99 \times 7.0 - 5.0 = 1.0 + 6.93 - 5.0 = 2.93$$

第 2 步：计算 Actor 损失

$$\pi(a_1|s) = \text{softmax}([0.5, 1.5])[1] = e^{1.5} / (e^{0.5} + e^{1.5}) = 0.731$$

$$\log \pi(a_1|s) = -0.313$$

$$\text{actor_loss} = -\log \pi(a_1|s) \times \delta = -(-0.313) \times 2.93 = 0.917$$

(注意: $\delta > 0$ 表示动作好, loss 为正, 梯度下降会增大 $\log \pi$)

第 3 步：计算 Critic 损失

$$\text{TD target} = r + \gamma \cdot V(s') = 1.0 + 0.99 \times 7.0 = 7.93$$

$$\text{critic_loss} = (V(s) - \text{TD target})^2 = (5.0 - 7.93)^2 = 8.585$$

第 4 步：计算熵正则

$$H = -\sum \pi \cdot \log \pi = -0.269 \times \log(0.269) - 0.731 \times \log(0.731) \approx 0.584$$

$$\text{entropy_loss} = -H = -0.584$$

第 5 步：总损失与更新

$$\text{total} = \text{actor_loss} + 0.5 \times \text{critic_loss} + 0.01 \times \text{entropy_loss}$$

$$= 0.917 + 0.5 \times 8.585 + 0.01 \times (-0.584) \approx 0.917 + 4.293 - 0.006 = 5.204$$

启示：Actor 与 Critic 协同——Critic 提供"动作有多好"的标量信号 δ , Actor 据此调整策略。 $\delta > 0$ 增大概率, $\delta < 0$ 减小概率。

8.1 用 δ_t 替代 G_t 降低了方差, 但引入新问题: δ_t 是单步估计, 依赖 V_ϕ 的准确性, V 估计有偏时 δ 也有偏。而 G_t 虽然方差大但无偏。如何在两者间权衡? Schulman 等人 2015 年提出的 GAE (Generalized Advantage Estimation) 用指数加权平均所有 n -step 优势:

第 1 步：定义 n -step 优势 (前 n 步真实 + 第 n 步的 V 估计)

$$A_t^{(n)} = r_{t+1} + \gamma r_{t+2} + \dots + \gamma^{n-1} r_{t+n} + \gamma^n V(s_{t+n}) - V(s_t)$$

第 2 步：GAE 用几何加权 $(\gamma\lambda)^k$ 把所有 n -step 优势合并

$$A_t^{\text{GAE}(\gamma, \lambda)} = \sum_{k=0}^{\infty} (\gamma\lambda)^k \delta_{t+k}$$

第 3 步：观察两个极端

· $\lambda=0$: $A_t = \delta_t$, 退化为单步 TD (偏差大、方差小)

- $\lambda=1$: $A_t = G_t - V(s_t)$, 退化为 MC (无偏、方差大)

实践中 $\lambda=0.95$ 几乎是黄金值, 在偏差与方差间取得最佳平衡。GAE 是 PPO、TRPO 等现代算法的标准组件, 几乎所有 on-policy 算法都使用它。

8.5 A2C 实现要点

配套代码 `ch08_a2c.py` 实现了完整的 A2C 算法。几个关键工程细节:

- **网络结构**: 输入 $\rightarrow$ 128 Tanh $\rightarrow$ 128 Tanh $\rightarrow$ (policy_head: Linear(action_dim) + value_head: Linear(1))
- **优化器**: Actor 与 Critic 用同一个 Adam 优化器 (学习率 $5e-4$)
- **rollout**: 每 5 步累积一次更新, 比单步稳定
- **梯度裁剪**: `clip_grad_norm_` 防止爆炸

运行该代码, 约 200-400 回合可让 CartPole-v1 达到 500 分。相比 DQN, A2C 训练曲线更平稳 (无 target network 的延迟), 但样本效率略低 (on-policy)。

A2C 解决了 REINFORCE 的方差与"必须等回合结束"问题, 但留下了一个**根本限制**:

on-policy——每个样本只用一次就丢弃。训练 100 万步需要 100 万个真实交互样本, 对机器人控制这类样本昂贵的场景太贵了。能否重复利用旧数据? 直接用旧数据训练新策略会"偏离太远"导致不稳定——下一章的 PPO 通过"限制策略更新幅度"实现"同批数据多次训练", 在不偏离 on-policy 安全性的前提下大幅提升样本效率, 成为现代 RL 的工业标准。

第九章 PPO: 现代 RL 工业标准

第八章末尾我们指出 A2C/REINFORCE 都是 on-policy——每个样本只用一次。训练需要海量真实交互, 对机器人控制这类样本昂贵的场景太昂贵了。**能否重复利用旧数据?** 直接用旧数据训练新策略会"偏离太远"导致不稳定——因为新策略 π_θ 与数据生成策略 $\pi_{\theta_{old}}$ 差异越大, 重要性采样的方差越大。

PPO (Proximal Policy Optimization, Schulman et al. 2017) 的核心思想: **限制策略更新幅度**, 安全地重复利用数据。它在 4-10 个 epoch 内重复使用同一批数据训练, 但通过"裁剪"防止策略偏离太远。OpenAI Five、ChatGPT 的 RLHF 阶段都用 PPO, 它已成为 RL 的"工业标准"。本章详细推导 PPO 的损失函数、超参数选择与完整实现。

9.1 从 TRPO 到 PPO

PPO 的思想继承自 TRPO (Trust Region Policy Optimization, 2015)。TRPO 的核心思想是"信任域约束": 每次更新限制新策略与旧策略的 **KL 散度** 不超过 δ (通常 0.01), 保证策略不会"变化过大"导致崩溃。数学上, TRPO 求解约束优化:

$$\max_{\theta} E[\pi_{\theta}(a|s)/\pi_{\theta_{old}}(a|s) \cdot A(s,a)] \text{ subject to } E[KL(\pi_{\theta_{old}}||\pi_{\theta})] \leq \delta$$

TRPO 用**共轭梯度法**求解这个约束优化, 实现复杂且计算开销大——每次更新需要计算 Fisher 信息矩阵、估计约束 Hessian-vector 乘积, 工程上不易实现。

PPO 的洞察：用简单的"裁剪"近似这个约束——不显式求解 KL 约束，而是直接把重要性采样比限制在 $[1-\epsilon, 1+\epsilon]$ 区间。这个简化让 PPO 实现简单（只需一行 clip）、训练高效（无需共轭梯度），同时保留了 TRPO 的稳定性。

9.2 PPO-Clip 损失函数

PPO 的推导分四步：

第 1 步：定义重要性采样比 $r_t(\theta)$

$$r_t(\theta) = \pi_{\theta}(a_t|s_t) / \pi_{\theta_{\text{old}}}(a_t|s_t)$$

旧数据来自 $\pi_{\theta_{\text{old}}}$ ，用新策略 π_{θ} 训练这些数据需要**重要性采样校正**—— r_t 就是新旧策略的概率比。 $r=1$ 表示新旧相同； $r>1$ 表示新策略更倾向该动作； $r<1$ 表示更不倾向。

第 2 步：写出朴素目标（直接最大化 $r \cdot A$ ）

$$L^{\text{naive}}(\theta) = E[r_t(\theta) \cdot A_t]$$

这本质上是策略梯度定理的重要性采样版本—— $A_t > 0$ 时希望增大 r （让新策略更倾向该好动作）， $A_t < 0$ 时希望减小 r 。

第 3 步：识别问题—— r 太大不安全

朴素目标鼓励 r 无限增大（ $A > 0$ 时），但 r 越大意味着 π_{θ} 偏离 $\pi_{\theta_{\text{old}}}$ 越远——重要性采样的方差与 r 成正比， r 太大会让梯度估计极不稳定。这正是 TRPO 要约束 KL 散度的原因。

第 4 步：PPO 解法——裁剪 r 在 $[1-\epsilon, 1+\epsilon]$ （ ϵ 通常 0.2）

$$L^{\text{CLIP}}(\theta) = E[\min(r_t(\theta) \cdot A_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) \cdot A_t)]$$

分两种情况解释为什么 **min** 起到"约束"作用：

- $A_t > 0$ （好动作）：希望 r 增大。但 $r > 1+\epsilon$ 时被裁剪——clip 项 = $(1+\epsilon) \cdot A < r \cdot A$ ，min 选 clip 项，梯度不再奖励 r 继续增大。所以**"好动作被鼓励，但鼓励有上限"**。
- $A_t < 0$ （坏动作）：希望 r 减小。但 $r < 1-\epsilon$ 时被裁剪——clip 项 = $(1-\epsilon) \cdot A > r \cdot A$ （注意 $A < 0$ ，所以 $(1-\epsilon) \cdot A$ 是更小的负数绝对值），min 选 $r \cdot A$ ，梯度推动 r 减小。所以**"坏动作被惩罚，但惩罚有下限"**。

关键观察：min 起到"悲观估计"的作用——在"策略变化方向有利但已过大"时取更保守的值（停止奖励），在"策略变化方向不利"时保留修正梯度。这种"单向裁剪"等价于软 KL 约束，但实现简单——只需一行 `torch.clamp`。

9.3 PPO 完整目标

PPO 的总损失结合了策略损失、价值损失、熵正则三项：

$$L_{\text{PPO}} = L_{\text{CLIP}} - c_1 \cdot L_{\text{VF}} + c_2 \cdot S[\pi_{\theta}]$$

其中：

- $L_{VF} = (V_{\theta}(s) - V_{\text{target}})^2$ ，价值损失（也常加裁剪）
- $S[\pi_{\theta}]$ 是策略熵，鼓励探索
- $c_1 = 0.5, c_2 = 0.01$ 是经验值

9.4 PPO 训练流程

☑ 具体计算实例：PPO Clipped Surrogate 数值计算

设旧策略 $\log \pi_{\text{old}}(a|s) = -0.5$ ，新策略 $\log \pi_{\text{new}}(a|s) = -0.3$ 。

优势 $A = +2.0$ （好动作）， $\epsilon = 0.2$ 。

第 1 步：计算重要性采样比 r

$$r = \exp(\log \pi_{\text{new}} - \log \pi_{\text{old}}) = \exp(-0.3 - (-0.5)) = \exp(0.2) \approx 1.221$$

第 2 步：计算 surr1 和 surr2

$$\text{surr1} = r \times A = 1.221 \times 2.0 = 2.442$$

$$\text{surr2} = \text{clip}(r, 1-\epsilon, 1+\epsilon) \times A = \text{clip}(1.221, 0.8, 1.2) \times 2.0 = 1.2 \times 2.0 = 2.4$$

第 3 步：取 $\min$ （保守目标）

$$L_{\text{CLIP}} = \min(\text{surr1}, \text{surr2}) = \min(2.442, 2.4) = 2.4$$

→ 因 $r = 1.221 > 1 + \epsilon = 1.2$ ，被裁剪。新策略虽更好，但不再奖励。

对比：若 $A = -2.0$ （坏动作）

$$\text{surr1} = 1.221 \times (-2.0) = -2.442$$

$$\text{surr2} = 1.2 \times (-2.0) = -2.4$$

$$L_{\text{CLIP}} = \min(-2.442, -2.4) = -2.442$$

→ 不裁剪！因 $r > 1$ 且 $A < 0$ 时， $\min$ 选更负的 surr1 ，梯度会推动 r 减小（降低坏动作概率）。

再对比：若 $r = 0.7, A = +2.0$

$$\text{surr1} = 0.7 \times 2.0 = 1.4$$

$$\text{surr2} = \text{clip}(0.7, 0.8, 1.2) \times 2.0 = 0.8 \times 2.0 = 1.6$$

$$L_{\text{CLIP}} = \min(1.4, 1.6) = 1.4$$

→ 不裁剪！因 $r < 1 - \epsilon = 0.8$ 且 $A > 0$ 时， $\min$ 选 surr1 ，梯度推动 r 增大（提升被低估的好动作）。

总结：PPO clip 只在"策略变化方向有利但已过大"时裁剪，保留"修正方向"的梯度。这是 PPO 稳定性的核心。

PPO 的训练流程与 A2C 类似，但有几个关键差异：

- 1. 用 $\pi_{\theta_{old}}$ 收集一批数据（通常 2048 步）
- 2. 计算 GAE 优势与回报
- 3. 在**同一批数据上做多次 epoch 更新**（通常 4-10 个 epoch）
- 4. 每个 epoch 内分 mini-batch（如 64 样本）更新
- 5. 更新后 $\pi_{\theta_{old}} \leftarrow \pi_{\theta}$ ，开始下一轮收集

第 3 步是 PPO 与 A2C 的本质区别：A2C 每批数据只用一次就丢弃，PPO 重复利用多次。裁剪机制保证了"重复利用"不会导致策略偏离数据生成分布太远——当 r 超出 $[1-\epsilon, 1+\epsilon]$ 时梯度被截断，自动停止"过度偏离"。

```
# PPO 核心更新: clipped surrogate + multi-epoch

# 关键创新：同一批数据训练多个 epoch
for epoch in range(4): # 4 个 epoch
    for mb_idx in mini_batches: # 分 mini-batch
        # 计算新策略的 log  $\pi_{new}(a|s)$ 
        logits = policy(states[mb_idx])
        new_log_prob = Categorical(logits=logits).log_prob(actions[mb_idx])

        # 重要性采样比  $r = \pi_{new} / \pi_{old}$ 
        #  $\exp(\log_{new} - \log_{old}) = \pi_{new} / \pi_{old}$ 
        ratio = (new_log_prob - old_log_probs[mb_idx]).exp()

        # — PPO Clip 损失 —
        # surr1: 原始目标
        # surr2: 裁剪后的目标 (限制  $r$  在  $[1-\epsilon, 1+\epsilon]$ )
        surr1 = ratio * advantages[mb_idx]
        surr2 = torch.clamp(ratio, 1-eps, 1+eps) * advantages[mb_idx]
        # 取 min: 保守更新, 防止策略变化过大
        actor_loss = -torch.min(surr1, surr2).mean()

        # — Value 损失 (也加裁剪) —
        v = critic(states[mb_idx])
        v_clipped = returns[mb_idx] + torch.clamp(v - returns[mb_idx], -eps, eps)
        value_loss = torch.max((v - returns[mb_idx])**2,
                                (v_clipped - returns[mb_idx])**2).mean()

        # 总损失 = actor + 0.5*value - 0.01*entropy
        loss = actor_loss + 0.5 * value_loss - 0.01 * entropy
```

9.5 超参数选择

超参数	推荐值	说明
学习率	3e-4	Adam; 线性衰减到 0 可提升后期稳定
clip ϵ	0.2	过大不稳, 过小学不到
epochs/batch	4	Atari 大状态可 10
mini-batch size	64	通常 32-256

超参数	推荐值	说明
steps per iter	2048	越大越稳但越慢
γ	0.99	默认
GAE λ	0.95	偏差-方差平衡
entropy coef	0.01	Atari 可 0.005
value coef	0.5	常用

表 9.1: PPO 推荐超参数 (CartPole/MuJoCo 通用)

9.6 PPO 的工程实践

配套代码 `ch09_ppo.py` 是完整实现。几个工程要点：

- **obs 标准化**：维护运行均值与方差，对 obs 做 $(x - \text{mean}) / \text{std}$ 。MuJoCo 必备，CartPole 可选
- **reward scaling**：除以运行 std，避免奖励幅度差异
- **优势标准化**：每批 $\text{adv} = (\text{adv} - \text{mean}) / (\text{std} + 1e-8)$
- **学习率衰减**：后期降低 lr 可进一步提升
- **正交初始化**：网络权重用 `nn.init.orthogonal_`，提升稳定性

[技巧] PPO 在 CartPole 上 100-200 个 iteration (约 200K 步) 可稳定达到 500 分。在 MuJoCo 上需 1M-10M 步，建议用 Gymnasium 的 vectorized env 加速。

PPO 是 on-policy 算法的**天花板**——稳定、高效、易实现，已成为 RLHF 与游戏 AI 的事实标准。但它仍有一个无法逾越的根本限制：**on-policy 数据**——即使可以重复训练多个 epoch，数据本身仍必须来自当前策略，样本效率有上限。

要进一步突破样本效率，必须转向**off-policy**——允许使用任意旧策略的数据。下一章的 SAC (Soft Actor-Critic) 正是为此而生：通过**"最大熵目标 + 双 Q 网络 + 自动温度系数"**三大设计，在连续控制上同时实现**高样本效率 (off-policy)** 与 **高稳定性 (最大熵正则)**，成为现代连续控制 RL 的工业标准。

第十章 SAC 与连续控制

第九章的 PPO 是 on-policy 算法的**"天花板"**——它在 Atari、MuJoCo、机器人控制等任务上稳定可靠，是工业界默认的 RL 算法。但 PPO 有两个根本性局限：

- **局限一：样本效率有上限**。on-policy 要求**"采一批、用一次、扔掉"**，每个样本只能参与一次梯度更新。若环境模拟代价高（如真实机器人），训练成本不可承受。
- **局限二：探索不够智能**。PPO 虽能处理连续动作，但探索依赖高斯分布的随机采样——采样分布与策略目标完全分离，无法根据**"探索是否有效"**自适应调整。

Soft Actor-Critic (SAC, Haarnoja et al. 2018) 针对这两个局限给出答案：转向 **off-policy**（样本可重复使用，提升样本效率），同时用**最大熵原理**让探索变成**"目标的一部分"**

——策略不再被动地随机，而是主动权衡"高回报"与"高熵"，从而实现更高效的探索。SAC 在 MuJoCo、Pendulum、机器人控制等连续任务上全面超过 PPO 与 DDPG，成为连续控制的 SOTA。

10.1 最大熵 RL：把探索写进目标函数

本节回答一个核心问题：如何让"探索"从被动手段变成主动目标？

10.1.1 标准 RL 的目标

标准 RL 的目标是最大化折扣期望回报：

$$J(\pi) = E_{\tau \sim \pi} [\sum_{t=0}^{\infty} \gamma^t r_t]$$

这里策略 π 只关心回报——它学到的就是"在状态 s 下选哪个动作 a 能让未来回报最大"。这种目标隐含一个倾向：策略最终会走向确定性（在每一步只选最高 Q 的动作），于是探索只能靠外加机制（ ϵ -greedy、高斯噪声等）被动维持。

10.1.2 加入熵奖励：让"随机性"成为奖励

最大熵 RL 的核心想法：在每一步的奖励里加上策略熵 $H(\pi(\cdot|s_t))$ ，让"保持随机"本身就得到奖励。熵的定义为：

$$H(\pi(\cdot|s)) = - \sum_a \pi(a|s) \log \pi(a|s)$$

把熵加入目标函数（一步一步推导）：

第 1 步：标准目标加上熵奖励项：

$$J_{\text{max-ent}}(\pi) = E_{\tau \sim \pi} [\sum_t \gamma^t (r_t + \alpha \cdot H(\pi(\cdot|s_t)))]$$

第 2 步：把熵 H 展开为 $-E_{a \sim \pi}[\log \pi]$ ：

$$J_{\text{max-ent}}(\pi) = E_{\tau \sim \pi} [\sum_t \gamma^t (r_t + \alpha \cdot E_{a_t \sim \pi} [-\log \pi(a_t|s_t)])]$$

第 3 步：合并期望，得到每步的"有效奖励"为 $r_t - \alpha \log \pi(a_t|s_t)$ ：

$$J_{\text{max-ent}}(\pi) = E_{\tau \sim \pi} [\sum_t \gamma^t (r_t - \alpha \log \pi(a_t|s_t))]$$

其中 α 是温度系数，控制"熵奖励"相对于"回报"的权重。直观上：智能体不仅想获得高回报，还想"尽可能随机地"获得高回报——这种随机性带来更强的探索能力和对环境变化的鲁棒性。

10.1.3 为什么这样做有效？

熵大 = 策略随机 = 探索多。最大熵 RL 让策略在"高回报"和"高随机"之间主动平衡：

- 若 $\alpha \rightarrow 0$ ：熵奖励可忽略，退化成标准 RL（贪婪、不探索）
- 若 $\alpha \rightarrow \infty$ ：熵奖励主导，策略变成均匀分布（光探索、不利用）
- 合适的 α ：策略在保证高回报的同时保留必要随机性，自然形成"该确定时确定、该探索时探索"的行为

10.1.4 推导最大熵 Bellman 方程

标准 RL 的 Bellman 方程为 $Q^\pi(s,a) = E[r + \gamma E_{a' \sim \pi}[Q^\pi(s',a')]]$ 。在最大熵框架下，"未来收益"不仅包含 Q 值，还包含下一步的熵奖励。一步步推导：

第 1 步：把 10.1.2 中的有效奖励代入 Q 的递归定义：

$$Q^\pi(s,a) = E_{s',r}[r + \gamma \cdot E_{a' \sim \pi}[Q^\pi(s',a') + \alpha \cdot H(\pi(\cdot|s'))]]$$

第 2 步：把熵展开成期望形式 $H(\pi) = -E[\log \pi]$ ：

$$Q^\pi(s,a) = E_{s',r}[r + \gamma \cdot E_{a' \sim \pi}[Q^\pi(s',a') - \alpha \log \pi(a'|s')]]$$

解读：在 s' 处，策略不仅关心 Q 值，还因 $\log \pi(a'|s')$ 项被"惩罚"——选高概率动作（ $\log \pi$ 大）被惩罚得多，选低概率动作（ $\log \pi$ 小）被惩罚得少，于是策略被推向"分散采样"。这正是最大熵框架把探索"硬编码"进价值函数的方式。

10.2 SAC 三大创新：把最大熵 RL 落地

直接实现最大熵 RL 会遇到三个工程难题，SAC 用三大创新一一化解。

10.2.1 创新 1：双 Q 网络取 $\min$ ——抑制 Q 值过估

问题：在连续控制中， Q 网络的过估问题比离散更严重——动作空间连续， $\max$ 操作放大噪声，配合 bootstrap 易正反馈发散。第 11 章会看到 Double DQN 解决离散过估；这里 SAC 借鉴 TD3 的思路，用两个独立 Q 网络取 $\min$ 。

做法：维护两个 Q 网络 Q_1, Q_2 ，目标值用 $\min$ 抑制过估：

$$Q_target = r + \gamma (\min(Q_1', Q_2') - \alpha \log \pi(a'|s'))$$

为什么取 $\min$ 能抑制过估？两个 Q 网络独立初始化、独立训练，它们同时对同一 (s,a) 高估的概率远小于单个网络高估的概率。取 $\min$ 自然把过估的部分"剪掉"，但不会影响低估——这正是我们想要的"乐观估计偏少"特性。

10.2.2 创新 2：自动温度系数 α ——免手调

问题： α 的选择很难——太大策略太随机，太小探索不足；而且不同任务、不同训练阶段的最佳 α 都不一样，手调极不现实。

做法：把 α 看作可学习参数，让策略熵自动接近"目标熵" H_{target} （通常设为 $-\dim(A)$ ，如连续 3 维动作作用 -3）。一步步推导：

第 1 步：写出约束条件——策略熵应等于目标熵：

$$E_{s,a \sim \pi}[-\log \pi(a|s)] = H_{target}$$

第 2 步：把约束转化为 α 的无约束优化目标（拉格朗日对偶形式）：

$$J(\alpha) = E_{s,a \sim \pi}[-\alpha \log \pi(a|s) - \alpha \cdot H_{target}] = E_{s,a \sim \pi}[-\alpha (\log \pi(a|s) + H_{target})]$$

第 3 步：对 $J(\alpha)$ 关于 α 求导（注意 $\log \pi$ 不依赖 α ）：

$$\partial J(\alpha) / \partial \alpha = -E[\log \pi(a|s) + H_{\text{target}}] = E[-\log \pi(a|s)] - H_{\text{target}} = H - H_{\text{target}}$$

第 4 步：梯度下降更新 α （学习率 η_α ）：

$$\alpha \leftarrow \alpha - \eta_\alpha \cdot \partial J / \partial \alpha = \alpha - \eta_\alpha \cdot (H - H_{\text{target}})$$

自适应规律：当熵 $H > H_{\text{target}}$ （探索太多）， $(H - H_{\text{target}}) > 0$ ， α 减小（少奖励熵，抑制探索）；反之 $H < H_{\text{target}}$ （探索太少）， $(H - H_{\text{target}}) < 0$ ， α 增大（多奖励熵，鼓励探索）。于是 α 在整个训练过程自动适配，无需人工介入。

10.2.3 创新 3：重参数化技巧——让采样可微

问题：SAC 的策略是高斯分布 $\pi(a|s) = N(\mu(s), \sigma(s))$ 。直接采样 $a \sim N(\mu, \sigma)$ 是一个随机操作，无法对 (μ, σ) 求导——但策略梯度需要把“采样后的回报”通过 a 传回到策略网络参数 (μ, σ) 。

做法：把随机性抽出来作为外部噪声 $\varepsilon \sim N(0, 1)$ ，让 a 表示为 μ, σ 的可微函数：

$$a = \tanh(\mu(s) + \sigma(s) \cdot \varepsilon), \varepsilon \sim N(0, 1)$$

现在 a 关于 (μ, σ) 可微，梯度可通过 a 流回策略网络。 $\tanh$ 限制 a 到 $[-1, 1]$ ，配合环境动作范围缩放即可覆盖任意连续动作空间。同时还要修正 $\log \pi(a|s)$ （因为 $\tanh$ 会改变分布形状，需加一个对数雅可比修正项 $\log(1 - \tanh^2(\cdot))$ ）。

10.3 SAC 完整算法

下面是 SAC 的核心伪代码，对齐前述三大创新：

```

# SAC 主循环：最大熵 RL + 双 Q + 自动  $\alpha$  + 重参数化
for step in range(num_steps):
    # — 1. 采样动作（重参数化技巧，创新 3）—
    #  $a = \tanh(\mu + \sigma \cdot \epsilon)$ ,  $\epsilon \sim N(0,1)$ , 使采样可微
    a, log_pi = policy.sample(state)
    next_state, r, done = env.step(a)
    buffer.push(state, a, r, next_state, done)
    state = next_state

    # — 2. 更新双 Q 网络（创新 1：取 min 抑制过估）—
    batch = buffer.sample(256)
    with torch.no_grad():
        a_next, log_pi_next = policy.sample(s_next)
        # TD 目标含熵奖励:  $r + \gamma(\min Q' - \alpha \cdot \log \pi)$ 
        q_target = r + gamma * (min(Q1'(s_next, a_next),
                                   Q2'(s_next, a_next))
                               - alpha * log_pi_next)
        q1_loss = mse(Q1(s, a), q_target)
        q2_loss = mse(Q2(s, a), q_target)

    # — 3. 更新策略（创新 3：重参数化使梯度可传回）—
    a_new, log_pi_new = policy.sample(s)
    q_new = min(Q1(s, a_new), Q2(s, a_new))
    # 目标：最大化  $Q - \alpha \cdot \log \pi$ （回报 + 熵）
    policy_loss = (alpha * log_pi_new - q_new).mean()

    # — 4. 自动温度系数  $\alpha$ （创新 2）—
    # 让熵接近 target_entropy（通常  $-\dim(A)$ ）
    alpha_loss = -(log_alpha * (log_pi_new + target_entropy).detach()).mean()

    # — 5. 软更新目标网络（Polyak 平均）—
    #  $\theta_{\text{target}} \leftarrow \tau \cdot \theta + (1-\tau) \cdot \theta_{\text{target}}$ ,  $\tau=0.005$ 
    soft_update(Q1_target, Q1, tau=0.005)
    soft_update(Q2_target, Q2, tau=0.005)

```

10.4 SAC 在 Pendulum 上的表现

☒ 具体计算实例：SAC 单步更新数值演示

设连续动作任务， $\gamma=0.99$, $\alpha=0.2$ （自动调节中）。

当前状态 s ，策略采样 a ，得 $r=1.0$ ，转到 s' ， $\text{done}=\text{False}$ 。

两个 Q 网络输出： $Q_1(s,a)=4.0$, $Q_2(s,a)=4.5$ （取 $\min=4.0$ ）

在 s' 处： $Q_1(s',a')=5.0$, $Q_2(s',a')=6.0$, $\log \pi(a'|s')=-0.4$

第 1 步：计算 TD 目标（含熵奖励，对应 10.1.4 的 Bellman 方程）

$$q_{\text{next}} = \min(Q_1(s',a'), Q_2(s',a')) - \alpha \cdot \log \pi(a'|s')$$

$$= \min(5.0, 6.0) - 0.2 \times (-0.4) = 5.0 + 0.08 = 5.08$$

$$q_{\text{target}} = r + \gamma \cdot q_{\text{next}} = 1.0 + 0.99 \times 5.08 = 6.029$$

第 2 步：Q 网络损失（取 min 的 Q_1 被更新，创新 1）

$$q1_loss = (Q_1(s,a) - q_target)^2 = (4.0 - 6.029)^2 = 4.117$$

$$q2_loss = (Q_2(s,a) - q_target)^2 = (4.5 - 6.029)^2 = 2.340$$

第 3 步：策略损失（重参数化采样 a_new ，创新 3）

$$q_new = \min(Q_1(s,a_new), Q_2(s,a_new)) = 4.2 \text{（假设）}$$

$$\log \pi(a_new|s) = -0.5 \text{（假设）}$$

$$policy_loss = \alpha \cdot \log \pi - q_new = 0.2 \times (-0.5) - 4.2 = -4.3$$

（最小化 -4.3 = 最大化 $q_new - \alpha \cdot \log \pi$ ，即回报高且熵大）

第 4 步：自动 α 更新（target_entropy = -1，创新 2）

当前熵 $H = -\log \pi \approx 0.5$ (< target 1.0，熵太小)

$$alpha_loss = -\log_alpha \times (\log \pi + target_entropy)$$

$$= -\log_alpha \times (-0.5 + (-1)) = -\log_alpha \times (-1.5)$$

最小化 $alpha_loss \rightarrow \log_alpha$ 增大 $\rightarrow \alpha$ 增大

$\rightarrow$ 更强的熵奖励 $\rightarrow$ 策略变得更随机（增大熵）

启示：SAC 的三个组件形成自调节系统——Q 抑制过估、策略追求“高回报+高熵”、 α 自动维持目标熵。

配套代码 `ch10_sac.py` 在 Pendulum-v1 上训练。Pendulum 是经典连续控制任务：一根杆子，可以用 $[-2, 2]$ 范围的力矩使其立起。奖励是 $-\theta^2 - 0.1 \cdot (\text{action})^2$ ，最大约 0，最差评约 -16。SAC 通常 20K 步内可将平均奖励从 -1600 提升到 -200 左右，比 DDPG/TD3 更稳。

10.5 SAC vs PPO vs DDPG/TD3

算法	on/off	动作空间	样本效率	稳定性	适用场景
PPO	on-policy	离散/连续	低	高	通用，并行训练
SAC	off-policy	连续	高	高	连续控制首选
TD3	off-policy	连续	高	中	比 DDPG 稳定
DDPG	off-policy	连续	高	低	已淘汰
DQN	off-policy	离散	中	中	离散控制

表 10.1：主流深度 RL 算法对比

[技巧] SAC 的优势在样本效率：在 MuJoCo HalfCheetah 上，SAC 100K 步可达 DDPG 1M 步的效果。若你的任务环境模拟代价高（如真实机器人），SAC 是不二之选。

本章小结与下一章预告

SAC 通过"最大熵 RL + off-policy + 双 Q + 自动 α + 重参数化"五件套，解决了 PPO 在连续控制上的样本效率与探索难题，成为连续控制的 SOTA。

但**离散控制**呢？第六章 DQN 仍是主力，但 DQN 本身有几个已知问题——Q 值过估（max 操作放大偏差）、V/A 不分离（状态价值与动作优势混在一起）、样本采样不均（均匀采样浪费高信号样本）。下一章**Rainbow DQN**整合 6 项改进，把这些遗留问题逐一化解，把 DQN 推到极致。

第十一章 Rainbow DQN 与改进技巧

第六章 DQN 在 Atari 上取得突破，但留下了四个已知问题：

- **问题 1：Q 值过估。**TD 目标用 $\max_a Q_{\text{target}}(s', a')$ ，max 操作放大偏差，配合 bootstrap 易正反馈发散。
- **问题 2：V/A 不分离。** $Q(s, a)$ 同时编码"状态好坏 $V(s)$ "与"动作优势 $A(s, a)$ "，但很多状态本身的好坏比动作选择更重要——网络被动作选择干扰。
- **问题 3：样本采样不均。**均匀采样让所有转移等概率被选，但 TD 误差大的样本蕴含更多学习信号，被均匀稀释掉。
- **问题 4：探索不足。** ϵ -greedy 在稀疏奖励任务上难以发现新状态。

本章介绍前三项的解决方案——Double DQN、Dueling Network、Prioritized Experience Replay。这三项与 Multi-step Returns、Distributional RL、NoisyNet 一起，构成 Rainbow DQN (Hessel et al. 2018)，在 Atari 57 游戏上大幅超越原始 DQN，是值方法的高峰。

11.1 Double DQN：解决 Q 值过估（问题 1）

问题回顾：原始 DQN 的 TD 目标为：

$$Q_{\text{target}} = r + \gamma \cdot \max_a Q_{\text{target}}(s', a')$$

这里 max 操作同时承担两个角色——"选动作"和"估价"。如果 $Q_{\text{target}}(s', a')$ 对某个动作 a' 高估了，max 会选中这个动作，TD 目标被这个高估值主导，越学越大。

举例：假设 s' 处 $Q_{\text{target}}(s', \cdot) = [2.5, 4.5, 6.0]$ ，但动作 2 的真实值只有 4.0（高估 2.0）。max 选中动作 2， $q_{\text{next}} = 6.0$ ；如果选中动作 1（真实值 4.5）， $q_{\text{next}} = 4.5$ 。正误差被 max "锁定"，无法被纠正。

Double DQN 解法 (van Hasselt et al. 2015)：把"选动作"和"估价"拆开，由两个网络分别承担。一步步推导：

第 1 步：原始 DQN 的做法（选 + 估都用 target_net）：

$$a^* = \operatorname{argmax}_a Q_{\text{target}}(s', \cdot), q_{\text{next}} = Q_{\text{target}}(s', a^*)$$

第 2 步：拆开两个角色——用 policy_net 选动作（"我认为哪个动作好"）：

$$a^* = \operatorname{argmax}_a Q_{\text{policy}}(s', \cdot)$$

第 3 步：用 target_net 估价（"客观上值多少"）：

$$q_{\text{next}} = Q_{\text{target}}(s', a^*)$$

第 4 步：得到 Double DQN 的 TD 目标：

$$Q_{\text{target}} = r + \gamma \cdot Q_{\text{target}}(s', \operatorname{argmax}_a Q_{\text{policy}}(s', a'))$$

为什么有效：两个网络独立训练，policy_net 对 a^* 的高估未必对应 target_net 的高估——选错动作的高估与估值的高估不再耦合，期望意义下的过估大幅降低。实现只需改动一行代码：

```
# — 原始 DQN: 选动作 + 估价都用 target_net —
# 问题: max 操作放大过估误差
q_next = target_net(s_next).max(dim=1)[0]

# — Double DQN: 分工合作 —
# 1. 用 policy_net 选动作 (我认为哪个动作好)
a_next = policy_net(s_next).argmax(dim=1, keepdim=True)
# 2. 用 target_net 估价 (客观上值多少)
# 这样过估误差被打破相关性, 显著降低
q_next = target_net(s_next).gather(1, a_next).squeeze(1)
```

11.2 Dueling Network: 分离 V 与 A (问题 2)

问题回顾：DQN 的网络直接输出 $Q(s, a)$ ——同时编码"状态本身好坏 $V(s)$ "和"动作相对优势 $A(s, a)$ "。但很多状态下动作选择无关紧要（如杆子已经稳定平衡，怎么动都一样），此时网络却仍要为每个动作更新 Q 值，浪费信号并引入噪声。

Dueling DQN 解法 (Wang et al. 2015)：改变网络结构，把 $Q(s, a)$ 显式分解为 $V(s) + A(s, a)$ 。一步步推导：

第 1 步：从 Q 的定义出发。 $Q(s, a) = V(s) + A(s, a)$ ，其中：

$$V(s) = E_{a \sim \pi}[Q(s, a)], A(s, a) = Q(s, a) - V(s)$$

$V(s)$ 是状态本身的价值， $A(s, a)$ 是动作 a 相对平均的"优势"。

第 2 步：直接写 $Q = V + A$ 有"不可识别性"问题—— V 和 A 可任意加减常数 c ： $V+c$ 与 $A-c$ 给出相同的 Q 。网络无法唯一确定 V 、 A 的值，训练会震荡。

第 3 步：减均值约束。让优势 A 的均值为 0，迫使 V 唯一：

$$Q(s, a) = V(s) + (A(s, a) - \operatorname{mean}_a A(s, a'))$$

此时 $\operatorname{mean}_a A(s, a) = 0$ ，于是 $V(s) = \operatorname{mean}_a Q(s, a)$ ， V 与 A 都被唯一确定。这就是 Dueling 的最终形式。

第 4 步：网络结构。共享特征层后分两路：

- **value branch**：输出 $V(s)$ ，一个标量
- **advantage branch**：输出 $A(s, \cdot)$ ，动作维度向量
- 合并： $Q(s,a) = V(s) + A(s,a) - \text{mean}(A)$ ，再按 DQN 流程计算

效果：在"动作无关紧要"的状态上，网络只需更新 V 而 A 保持小值，学习状态价值更快、噪声更小，整体收敛速度提升 20-30%。

```
# Dueling DQN: 将  $Q(s,a)$  分解为  $V(s) + A(s,a)$ 
class DuelingQNetwork(nn.Module):
    def __init__(self, state_dim, action_dim):
        super().__init__()
        # 共享特征提取层
        self.feature = nn.Sequential(
            nn.Linear(state_dim, 128), nn.ReLU())
        # V 分支: 评估状态本身好坏
        self.value = nn.Sequential(
            nn.Linear(128, 128), nn.ReLU(),
            nn.Linear(128, 1)) #  $V(s)$ 
        # A 分支: 评估动作相对优势
        self.advantage = nn.Sequential(
            nn.Linear(128, 128), nn.ReLU(),
            nn.Linear(128, action_dim)) #  $A(s,a)$ 

    def forward(self, x):
        h = self.feature(x) # 共享特征
        v = self.value(h) #  $V(s)$  标量
        a = self.advantage(h) #  $A(s,a)$  向量
        # 合并:  $Q(s,a) = V(s) + A(s,a) - \text{mean}(A)$ 
        # 减均值保证可识别性 (否则  $V$  和  $A$  可任意加减常数)
        return v + (a - a.mean(dim=1, keepdim=True))
```

11.3 Prioritized Experience Replay (问题 3)

问题回顾：DQN 用均匀采样从回放缓冲区取样本，所有转移等概率被选中。但 TD 误差大的样本蕴含更多学习信号——它们代表"网络还没学好"的转移，均匀采样让这些高价值样本被大量"已知"样本稀释，学习效率低。

PER 解法 (Schaul et al. 2015)：按 |TD 误差| 比例采样，TD 误差大的样本被采样概率高。一步步推导：

第 1 步：定义样本优先级。用 $|\delta_i| + \epsilon$ 作为优先级 (ϵ 防止 0 优先级样本永不被采)：

$$p_i = |\delta_i| + \epsilon$$

第 2 步：定义采样概率。加指数 α 控制优先化程度 ($\alpha=0$ 退化为均匀, $\alpha=1$ 完全优先)：

$$P(i) = p_i^\alpha / \sum_k p_k^\alpha$$

第 3 步：但优先采样改变了数据分布，会引入偏差——高 $|\delta|$ 样本被频繁采样，等于"过拟合"这部分数据。需要**重要性采样校正** (IS weight)：

$$w_i = (1 / (N \cdot P(i)))^\beta$$

其中 N 是缓冲区大小, $P(i)$ 是采样概率。直观上: $P(i)$ 大(被频繁采样)的样本 w 小(权重低), 抵消"过采样"的影响; $P(i)$ 小的样本 w 大, 鼓励"罕见但正确"的更新。

第 4 步: β 退火。完全校正需要 $\beta=1$, 但训练初期样本质量不稳定, 完全校正反而引入噪声。让 β 从 0.4 线性退火到 1.0:

β : 0.4 $\rightarrow$ 1.0 (线性退火, 训练末期完全无偏)

效果: PER 在稀疏奖励任务上提升显著——TD 误差大的"关键转移"(如得分瞬间)被反复利用, 加速价值传播。代价是缓冲区实现复杂度增加(用 SumTree 维护概率)。

11.4 其他三项改进与数值实例

☒ 具体计算实例: Double DQN vs DQN 过估问题

设 s' 处两个网络输出(带噪声):

$\text{policy_net}(s') = [3.0, 5.0, 4.0]$ (动作 0/1/2)

$\text{target_net}(s') = [2.5, 4.5, 6.0]$ (target 略高, 含过估)

当前转移 $r=1, \gamma=0.99, \text{done}=\text{False}$ 。

原始 DQN (用 target_net 选 + 估):

$a^* = \arg\max \text{target_net}(s') = 2$ (值 6.0)

$q_{\text{next}} = \text{target_net}(s')[2] = 6.0$

$q_{\text{target}} = 1 + 0.99 \times 6.0 = 6.94$

Double DQN (policy 选, target 估):

$a^* = \arg\max \text{policy_net}(s') = 1$ (值 5.0)

$q_{\text{next}} = \text{target_net}(s')[1] = 4.5$ (注意: 取动作 1, 不是动作 2)

$q_{\text{target}} = 1 + 0.99 \times 4.5 = 5.455$

差异: DQN $q_{\text{target}}=6.94$, Double DQN $q_{\text{target}}=5.455$

DQN 因 target_net 对动作 2 过估, 导致 q_{target} 虚高 1.485

Double DQN 通过"用另一个网络选动作"打破相关性, 抑制过估 ✓

PER 采样权重实例:

设缓冲区有 4 个样本，TD 误差分别为 $[0.1, 0.5, 2.0, 0.05]$

优先级 $p = |\delta| + \epsilon = [0.1005, 0.5005, 2.0005, 0.055]$

$\alpha=0.6, P(i) = p^\alpha / \sum p^\alpha$

$P = [0.1005^{0.6}, 0.5005^{0.6}, 2.0005^{0.6}, 0.055^{0.6}] / \text{sum}$

$= [0.252, 0.661, 1.516, 0.169] / 2.598$

$\approx [0.097, 0.254, 0.584, 0.065]$

→ 样本 3 (TD=2.0) 被采样概率 58.4%，是样本 4 (TD=0.05) 的 9 倍

完整 Rainbow 还包含以下三项改进：

- **Multi-step Returns**：用 n-step TD 替代 1-step，平衡偏差与方差（n=3 常用）
- **Distributional RL (C51)**：学 Q 值的分布而非期望，更稳定
- **NoisyNet**：在网络中加入可学习噪声参数，替代 ϵ -greedy 探索（解决问题 4）

这六项改进在 Atari 上每一项都有显著提升，组合后达到 SOTA。实践中不一定全部使用——Double + Dueling + PER 是性价比最高的组合。

11.5 实战与对比

配套代码 `ch11_rainbow.py` 实现了 Double + Dueling + PER 三项组合。运行后对比原始 DQN (`ch06_dqn.py`)，可以观察到：

- 训练曲线更平稳（Double 抑制过估）
- 收敛速度更快 20-30%（Dueling 提高学习效率）
- 早期学习更高效（PER 聚焦高信号样本）

改进项	效果	实现复杂度	推荐
原始 DQN	基线	低	教学
+ Double	抑制过估，稳定训练	低（改 1 行）	强烈推荐
+ Dueling	收敛更快	中（改网络）	推荐
+ PER	样本效率 +20-30%	中（改 buffer）	推荐
+ Multi-step	方差-偏差平衡	低（改 target）	可选
+ C51	更稳定但复杂	高	研究用
+ NoisyNet	替代 ϵ -greedy	中	可选
= Rainbow	SOTA on Atari	高	生产

表 11.1: DQN 改进项的性价比分析

本章小结与下一章预告

Rainbow DQN 用 6 项改进逐一化解了 DQN 的过估、V/A 混淆、样本低效、探索不足四大问题，把值方法推向极致。至此，前 11 章覆盖了从动态规划到 DQN、从 REINFORCE 到 PPO、从 DDPG 到 SAC 的完整算法谱系——既有 on-policy 也有 off-policy，既有值方法也有策略方法，既有离散控制也有连续控制。

第 12 章展望**前沿方向**：RL 如何驱动大模型对齐（RLHF，把 PPO 用于 LLM）、世界模型（Dreamer，介于 DP 与 SAC 之间）、离线 RL（从固定数据集学习）、多智能体 RL（把 Actor-Critic 扩展到多智能体）。

第十二章 前沿主题：RLHF、DPO、世界模型、离线 RL、MARL

前 11 章覆盖了 RL 的“经典算法谱系”——从最简单的 Q-Learning 到工业级 PPO/SAC。本章覆盖 2020 年后的前沿方向，它们都在经典框架上“破除某个假设”——或把奖励来源从环境转向人类（RLHF/DPO），或把模型从无模型转向有模型（世界模型），或把数据从在线转向离线（离线 RL），或把智能体从单智能体转向多智能体（MARL）。

12.1 RLHF：驱动大模型对齐

与前面章节的关联：第九章 PPO 不只能打游戏——它也是 ChatGPT、Claude、GPT-4 等大模型对齐人类偏好的核心算法。RLHF（Reinforcement Learning from Human Feedback）让 PPO 走出了游戏世界，成为大模型时代最成功的 RL 应用。

其流程分三个阶段：

- **阶段一：SFT（Supervised Fine-Tuning）**：用高质量指令数据微调基座模型，让其学会“按指令生成”
- **阶段二：奖励模型 RM 训练**：收集人类对模型输出的偏好对 (y_w, y_l) ，训练一个 RM 模型，使其输出 $r(x, y)$ 反映人类偏好
- **阶段三：PPO 微调**：用 RM 作为奖励，PPO 优化 LLM 策略，同时加 KL 惩罚防止 LLM 偏离 SFT 模型太远

RLHF 的关键损失函数：

$$L_{\text{RLHF}} = E[r_{\text{RM}}(x, y)] - \beta * \text{KL}[\pi_{\theta} || \pi_{\text{SFT}}]$$

KL 项是 RLHF 的核心防御机制：没有它，LLM 可能“reward hacking”——学会输出 RM 给高分但实际无意义的内容。配套代码 `ch12_rlhf_concept.py` 用 toy 环境演示了这个机制。

RLHF 的三大痛点：**需要训练 RM、PPO 需要 4 个模型同时运行（显存大、调参难）、超参数多**。这些痛点催生了一个问题：能否跳过 RM 和 PPO，直接用偏好对训练策略？

12.2 DPO: 直接偏好优化

DPO (Direct Preference Optimization, Rafailov et al. NeurIPS 2023) 回答了上面的问题：可以。它证明了 RLHF 的最优解有闭式表达，跳过奖励模型和 PPO，一步完成对齐。这一步把"3 阶段 + 4 个模型"简化为"1 阶段 + 2 个模型"，数学上严格等价于 RLHF。

12.2.1 DPO 数学推导 (5 步)

从 RLHF 目标出发，5 步得到 DPO 损失。

第 1 步：写出 RLHF 优化目标

$$\max E[r(x,y)] - \beta * KL[\pi(\cdot|x) || \pi_{ref}(\cdot|x)]$$

第 2 步：求闭式最优策略 (拉格朗日乘子法)

$$\pi^*(y|x) = \pi_{ref}(y|x) * \exp(r(x,y)/\beta) / Z(x)$$

$Z(x)$ 是配分函数。直觉：奖励高的回答概率被放大，低的被压低。

第 3 步：反解奖励函数

$$r(x,y) = \beta * \log(\pi^*(y|x) / \pi_{ref}(y|x)) + \beta * \log Z(x)$$

关键洞察：奖励函数可用策略比值表达，不需要单独训练 RM。

第 4 步：代入 Bradley-Terry 偏好模型

$$P(y_w > y_l) = \sigma(r(x,y_w) - r(x,y_l))$$

代入反解结果， $Z(x)$ 在相减中消去（好/坏回答共享同一 prompt）：

$$P(y_w > y_l) = \sigma(\beta * \log(\pi(y_w)/\pi_{ref}(y_w)) - \beta * \log(\pi(y_l)/\pi_{ref}(y_l)))$$

第 5 步：DPO 损失函数 (取负对数似然)

$$L_{DPO} = -E[\log \sigma(\beta * \log(\pi_{\theta}(y_w)/\pi_{ref}(y_w)) - \beta * \log(\pi_{\theta}(y_l)/\pi_{ref}(y_l)))]$$

这就是 DPO 的全部！ 没有奖励模型、没有 PPO、没有价值网络。只需要：策略模型 + 参考模型 + 偏好对 (y_w, y_l) 。

[提示] 直觉：DPO 让好回答的 log 概率比参考模型高，坏回答的 log 概率比参考模型低。 β 控制调整力度——大则激进，小则温和。

12.2.2 DPO 完整实现

DPO 实现比 RLHF 简单得多：

```
# DPO 核心 PyTorch 实现
class DPOTrainer:
    def __init__(self, model_name, beta=0.1, lr=5e-7):
        self.beta = beta
        self.model = AutoModelForCausalLM.from_pretrained(model_name)
        self.ref_model = AutoModelForCausalLM.from_pretrained(model_name)
        self.ref_model.eval()
        for p in self.ref_model.parameters(): p.requires_grad = False
        self.optimizer = AdamW(self.model.parameters(), lr=lr)

    def dpo_loss(self, batch):
        pi_w = self.compute_log_probs(self.model, batch["chosen"])
        pi_l = self.compute_log_probs(self.model, batch["rejected"])
        with torch.no_grad():
            ref_w = self.compute_log_probs(self.ref_model, batch["chosen"])
            ref_l = self.compute_log_probs(self.ref_model, batch["rejected"])
        logits = self.beta * ((pi_w - ref_w) - (pi_l - ref_l))
        loss = -F.logsigmoid(logits).mean()
        return loss

    def train_step(self, batch):
        loss = self.dpo_loss(batch)
        self.optimizer.zero_grad(); loss.backward()
        clip_grad_norm_(self.model.parameters(), 1.0)
        self.optimizer.step()
        return loss.item()
```

12.2.3 DPO 变体：IPO 与 KTO

- **IPO (Identity PO)**：用 L2 损失替代 sigmoid 损失，防止过拟合。适合小数据集。
- **KTO (Kahneman-Tversky Optimization)**：不需要偏好对，只需"好/坏"标签。基于前景理论，数据需求减半。

12.2.4 RLHF vs DPO 对比

维度	RLHF (PPO)	DPO	IPO	KTO
阶段数	3 (SFT+RM+PPO)	1	1	1
模型数	4 (policy+ref+RM+value)	2 (policy+ref)	2	2
偏好数据	需要偏好对	需要偏好对	需要偏好对	只需好/坏标签
训练稳定性	低 (PPO 调参难)	高 (梯度下降)	高	高
推荐场景	大规模	通用首选	小数据集	只有好/坏标签

表 12.1 RLHF vs DPO vs IPO vs KTO 对比

12.2.5 DPO 实战建议

- **beta=0.1** 最常用；学习率比 SFT 低 10 倍
- **参考模型**：用 SFT 后的模型，不用原始预训练模型
- **早停**：监控 reward margin，停止增大后即停训

- **混合 SFT**：训练中混入少量 SFT 数据，防止遗忘

[提示] TRL 库 (huggingface.co/trl) 3 行代码启动 DPO 训练。Llama 3、Qwen 2、Mistral 等开源模型都用 DPO 对齐。

12.3 世界模型：Dreamer 系列

与前面章节的关联：第三章 DP 假设 MDP 模型已知（直接用模型递推），第十章 SAC 完全不学模型（无模型，靠环境交互学习）。世界模型在两者之间：学一个模型，但不完全用 DP——只在学的模型里“想象”经验，再用 actor-critic 训练。

Dreamer (Hafner et al. 2020) 系列采用“潜在空间世界模型”：在紧凑的潜在空间中学习环境动力学，再用该模型“想象”经验训练策略。流程：

- 1. 用 VAE 类编码器将图像观测编码到潜在向量 z_t
- 2. 在潜在空间学 RSSM (Recurrent State-Space Model)：预测下一潜在状态
- 3. 在模型中“想象”长轨迹，用 actor-critic 训练策略

DreamerV3 (2023) 在 150+ 任务上达到 SOTA，且无需超参数调整——这是基于模型 RL 的重大突破。

12.4 离线 RL：从固定数据集学习

与前面章节的关联：前 11 章都是在线 RL。离线 RL 从固定数据集学习，不需要交互——类似第四章 MC（从样本估计回报），但更难：数据来自其他策略，无法主动收集新数据。

在医疗、金融、自动驾驶等领域，在线试错代价高昂甚至危险。核心挑战是**分布偏移**：若学到的策略与数据集分布差异大，Q 值估计会发散。主要算法：

- **CQL (Conservative Q-Learning)**：在 Q 上加正则项，惩罚未见过的 (s,a) 对的 Q 值
- **IQL (Implicit Q-Learning)**：用 expectile regression 学 V，避免查询未见过的动作
- **Decision Transformer**：把 RL 转化为序列建模，用 Transformer 直接学 return-to-go $\rightarrow$ action 映射

12.5 多智能体 RL：MARL

与前面章节的关联：前 11 章都是单智能体。MARL 把第八章 Actor-Critic 扩展到多智能体：每个智能体是独立 actor-critic，但它们的环境（即其他智能体的策略）在不停变化，引入非平稳性问题。

当多个智能体共享环境时，每个智能体的最优策略依赖其他智能体的策略——这就进入了博弈论范畴。核心概念：

- **完全合作**：所有智能体共享奖励，目标一致。如机器人协作搬物。
- **完全竞争**：零和博弈。如围棋、星际争霸。

- **混合**：既有合作又有竞争。如 Dota 2 5v5。

主流算法：

- **Independent Q-Learning**：每个智能体独立学习，最简单但不稳定
- **MADDPG**：每个智能体学自己的 Q，但 Q 输入包含所有智能体动作（centralized training, decentralized execution）
- **MAPPO**：PPO 的多智能体扩展，OpenAI Five、AlphaStar 都用类似方法
- **HATPO**：基于 transformer 的多智能体，2022 年 SOTA

12.6 大模型时代的 RL 新方向

2023 年以来，大语言模型 + RL 出现许多新方向：

- **RLAIF**：用 AI（而非人类）提供反馈，替代 RLHF 中的人类标注
 - **Iterative DPO**：多轮 DPO——每轮用当前模型生成新偏好对，再训练。类似 Self-Play 思路
 - **Online DPO**：DPO + 在线采样——结合 RLHF 的探索性和 DPO 的简洁性
 - **Self-Play**：智能体与自己对弈，AlphaGo、AlphaZero 的核心思想
 - **In-context RL**：让 LLM 在推理时快速适应新任务，无需重新训练
- RL 与大模型的结合正在催生新一轮研究热潮。

第十三章 算法对比与工程实践

前面 12 章介绍了从 DP 到 SAC 的完整算法谱系——DP、MC、TD 是基础三件套；Q-Learning/DQN 是值方法主线；REINFORCE/A2C/PPO 是策略方法主线；DDPG/TD3/SAC 是连续控制主线；Rainbow 整合 DQN 改进；RLHF/世界模型/离线 RL/MARL 是前沿方向。本章总结这些算法的关系与选择指南——每个算法解决什么问题，适用于什么场景，并给出工程实践要点：训练曲线分析、超参数经验、复现论文的“潜规则”。

14.1 算法选择决策树

不同算法适用于不同场景，没有“万能算法”。下面的决策树总结了几十年的实践经验：

- **动作空间离散**？是 → 值方法（DQN 系列）；否 → 策略方法（PPO/SAC）
- **能否并行环境**？是 → PPO（并行是优势）；否 → SAC（数据效率高）
- **样本代价高**？是 → 基于模型方法（Dreamer）或离线 RL；否 → 无模型 OK
- **稀疏奖励**？是 → 加 intrinsic motivation / HER / 课程学习；否 → 标准 OK
- **多智能体**？是 → MAPPO 或 MADDPG；否 → 单智能体算法
- **需要在线学习**？是 → PPO（on-policy 适合连续适应）；否 → off-policy 可重用数据

14.2 算法综合对比

算法	类型	动作空间	样本效率	稳定	并行	实现难度	推荐度
Q-Learning	off-policy	离散	高	中	差	低	★★★★
DQN	off-policy	离散	中	中	中	中	★★★★★
Rainbow	off-policy	离散	中高	高	中	高	★★★★★
REINFORCE	on-policy	离散/连续	低	低	好	低	★★
A2C/A3C	on-policy	离散/连续	低	中	好	中	★★★★
PPO	on-policy	离散/连续	低中	高	好	中	★★★★★
DDPG	off-policy	连续	高	低	差	中	★★
TD3	off-policy	连续	高	中	差	中	★★★★★
SAC	off-policy	连续	高	高	差	中高	★★★★★

表 13.1: 主流 RL 算法综合对比

14.3 超参数经验

超参数	常见范围	经验法则
学习率 lr	1e-4 ~ 1e-3	Adam; 过大训练震荡, 过小收敛慢
折扣因子 γ	0.9 ~ 0.99	长期收益重要用 0.99; 短视任务可 0.9
批量大小 batch	32 ~ 256	越大越稳但越慢; GPU 显存限制
回放缓冲区	1e4 ~ 1e6	小任务 1e4 够; Atari 需 1e5+
目标网络同步	5 ~ 100 episodes	或用软更新 $\tau=0.005$
ϵ (探索)	0.05 ~ 0.3	从 1.0 指数衰减到 0.05
熵系数	0.001 ~ 0.05	探索不足时增大; Atari 0.01
GAE λ	0.9 ~ 0.97	默认 0.95
clip ϵ (PPO)	0.1 ~ 0.3	默认 0.2
τ (软更新)	0.001 ~ 0.01	SAC 0.005; TD3 0.005

表 13.2: RL 常用超参数经验值

14.4 训练曲线分析

如何判断训练是否成功？看奖励曲线是第一步，但还远远不够。一个完整的训练诊断应该包括：

- **奖励曲线**：是否上升？最终值？滑动平均是否稳定？
- **评估曲线**：定期用 `eval_mode`（无探索）评估，反映真实策略性能
- **损失曲线**：`actor_loss` / `critic_loss` / `value_loss` 是否收敛？
- **Q 值统计**：Q 值是否爆炸？均值是否合理？
- **熵曲线**：策略熵是否过早降到 0？
- **学习率**：是否按计划衰减？

[注意] 一个常见陷阱：奖励上升但 eval 性能下降——这通常是策略过拟合了特定探索路径。解决：用更大 buffer、更小 lr、或更严格的熵正则。

14.5 工程实践要点

13.5.1 复现论文的"潜规则"

RL 论文复现极难——同样的算法、同样的超参数，不同实现性能可能差几倍。主要原因：网络初始化、obs 标准化、reward scaling、梯度裁剪、优化器细节等"工程技巧"经常不在论文中描述。建议复现时：

- 参考 OpenAI baselines / stable-baselines3 / CleanRL 的实现
- 用相同的环境版本（Atari 有 multiple versions）
- 固定随机种子，重复 5+ 次取平均与方差
- 报告多 seed 结果，不只展示"最好"的曲线

13.5.2 推荐代码库

- **CleanRL**：单文件实现，最易学习与修改
- **Stable-Baselines3**：工业化封装，开箱即用
- **RLlib (Ray)**：分布式训练，生产级
- **DI-engine**：国产，中文文档完善
- **JAX 实现**（如 PureJaxRL）：GPU/TPU 加速，研究前沿

14.6 算法对比的可视化思维

最后，给读者一个直观的"算法地图"：

- **离散 vs 连续**：Q-Learning/DQN 处理离散；PPO/SAC 处理两者；DDPG/TD3 只连续
- **On vs Off policy**：PPO 是 on-policy 代表；SAC/DQN 是 off-policy 代表
- **基于值 vs 基于策略**：DQN 学 Q；REINFORCE/PPO 直接学 π ；AC 兼学
- **有模型 vs 无模型**：Dreamer 有模型；其他主流无模型
- **熵正则**：SAC 显式最大化熵；PPO 用熵 bonus；DQN 无显式熵

这个"五维分类"覆盖了 95% 的 RL 算法。读者在阅读新论文时，可以先定位它在每个维度上的位置，就能快速理解其本质。

第十四章 调试、调参与常见问题排查

强化学习训练常被称为"黑暗艺术"——即使算法正确，超参数不对也可能完全不收敛。本章总结作者在实践中遇到的高频问题及解决方案，帮助读者避开常见陷阱。建议在跑每个新算法时，先对照本章检查清单自查一遍。

14.1 训练不收敛：可能的原因

如果训练几百回合后奖励仍不上升，按以下顺序排查：

- **环境 API**：Gymnasium 0.26+ 的 5 元组返回值是否正确处理？reset 返回 (obs, info) 是否解包？
- **奖励范围**：奖励是否合理？是否需要 scaling（除以最大值）？
- **obs 标准化**：连续控制任务必须做；CartPole 等小状态可不做
- **学习率**：太大训练震荡，太小不收敛。先试 $1e-3$ ，不行降到 $1e-4$
- **网络结构**：太小可能欠拟合，太大可能过拟合。128-256 hidden 通常够
- **探索不足**：策略熵是否过早到 0？ ϵ 是否衰减太快？
- **批次太小**：尝试 batch_size=128 或 256
- **梯度爆炸**：加入梯度裁剪 clip_grad_norm_(5.0)

14.2 Q 值爆炸

DQN 类算法常见症状：训练初期 Q 值合理，但某时刻突然飙升到 $1e10$ 。原因：max 操作放大了过估误差，与 bootstrapping 形成正反馈。解决方案：

- 用 Double DQN（最有效）
- 减小学习率 ($1e-3 \rightarrow 3e-4$)
- 加大 batch_size ($64 \rightarrow 256$)
- 梯度裁剪 clip_grad_norm_(10.0)
- Huber loss 替代 MSE（对异常值更鲁棒）

14.3 策略熵过早坍缩

A2C/PPO 训练中常见：策略很快变成确定性，停止探索，性能卡在局部最优。诊断：监控策略熵曲线，若 50 回合内熵降到接近 0 即为此问题。解决方案：

- 增大熵系数 ($0.01 \rightarrow 0.05$)
- 降低学习率（防止策略变化过快）
- 增加网络容量 ($128 \rightarrow 256$)
- 检查 GAE λ 是否过大 (>0.97 方差大)

14.4 Gymnasium 版本兼容

Gymnasium 0.26 引入了 API 变更，许多旧代码无法直接运行。关键改动：

- `reset()` 返回 `(obs, info)`，而非 `obs`
- `step()` 返回 `(obs, reward, terminated, truncated, info)`，5 元组而非 4 元组
- `terminated`: 自然终止（任务完成或失败）
- `truncated`: 因时间限制等外部条件终止
- Bootstrap 时只用 `(1 - terminated)` 作 `mask`，不用 `truncated`

```
# Gymnasium 0.26+ 正确用法

# reset() 返回 (obs, info) 元组
obs, info = env.reset(seed=42)

for _ in range(max_steps):
    action = agent.act(obs)
    # step() 返回 5 元组（注意：不是 4 元组!）
    obs, reward, terminated, truncated, info = env.step(action)
    # done = 自然终止 OR 外部截断
    done = terminated or truncated

    # 重要：TD 更新时只用 terminated 作 mask
    # - terminated=True: 任务自然结束，不 bootstrap
    # - truncated=True: 时间到了，但任务未结束，应 bootstrap
    td_target = reward + gamma * Q(obs_next) * (1 - terminated)

    if done:
        break
```

14.5 GPU 显存爆炸

训练大网络时常见 OOM（Out of Memory）。排查与解决：

- 检查 `batch_size`，常见 64-256
- 检查回放缓冲区是否在 GPU 上（应在 CPU，采样时再传 GPU）
- 检查是否累积了计算图（`torch.no_grad()` / `detach()` 是否正确使用）
- 使用混合精度训练（`torch.cuda.amp`）
- 分布式训练用 `gradient accumulation` 模拟大 batch

14.6 探索-利用失衡

若训练曲线在低位震荡或卡住，可能是探索问题。诊断：

- **探索不足**：策略熵接近 0；不同 episode 行为几乎相同 → 加大探索
- **探索过度**：策略熵一直很高；eval 性能远高于 train → 减小探索

调节手段：

- ϵ -greedy: 调衰减速度，最终值 0.05
- 熵正则: PPO/SAC 的 `entropy_coef`

- NoisyNet: 替代 ϵ -greedy
- Intrinsic motivation: ICM/RND 给探索额外奖励
- Curiosity-driven: 用预测误差作为探索奖励

14.7 评估与对比的常见误区

报告 RL 结果时容易犯的错误:

- **只看一条曲线**: 必须多 seed (5+) 取平均, 画 shaded area (std)
- **报告 train 而非 eval**: eval 性能 (关闭探索) 才是策略真实性能
- **不公平对比**: 不同算法的环境数、总步数应一致
- **选最好 seed**: cherry-picking 是学术不端; 应报告所有 seed 的均值
- **不看样本效率**: 算法 A 在 1M 步达到 500 分, 算法 B 在 100K 步达到 450 分——B 在实际中更有用

14.8 调试 checklist

开始新算法时, 按以下 checklist 逐项检查:

- [] 环境正确加载, obs/action 维度对
- [] obs 标准化 (连续控制必备)
- [] reward 范围合理 (必要时 scaling)
- [] 网络初始化 (推荐 orthogonal_)
- [] 优化器 Adam, lr 合适
- [] 梯度裁剪已加
- [] entropy bonus 已加 (on-policy 算法)
- [] target 网络 / 软更新 (off-policy)
- [] buffer 容量合理
- [] 训练曲线、损失曲线、Q/熵曲线都在监控
- [] 多 seed 评估

[技巧] RL 调试经验积累极其重要。建议每实现一个算法, 记录下"踩过的坑"——下次复现或迁移到新任务时就能避免重蹈覆辙。这也是本教程反复强调"运行配套代码"的原因: 只有亲手跑过、亲手调过, 才能真正理解算法。

至此, 本教程的 14 章正文全部结束。从最基础的 MDP 到前沿的 RLHF, 从最简单的 Q-Learning 到工业级 PPO/SAC, 我们走过了强化学习的完整知识图谱。配套的 12 份代码可以立即运行——建议读者从第一章开始, 一边读代码一边对照本章理论, 遇到问题再回头查阅相关章节。强化学习的世界很大, 本教程只是入门; 但它提供的知识框架与代码原型, 足以支撑读者继续探索更深的研究方向与更复杂的实际应用。